مسارات الـ EMV في EasyLink

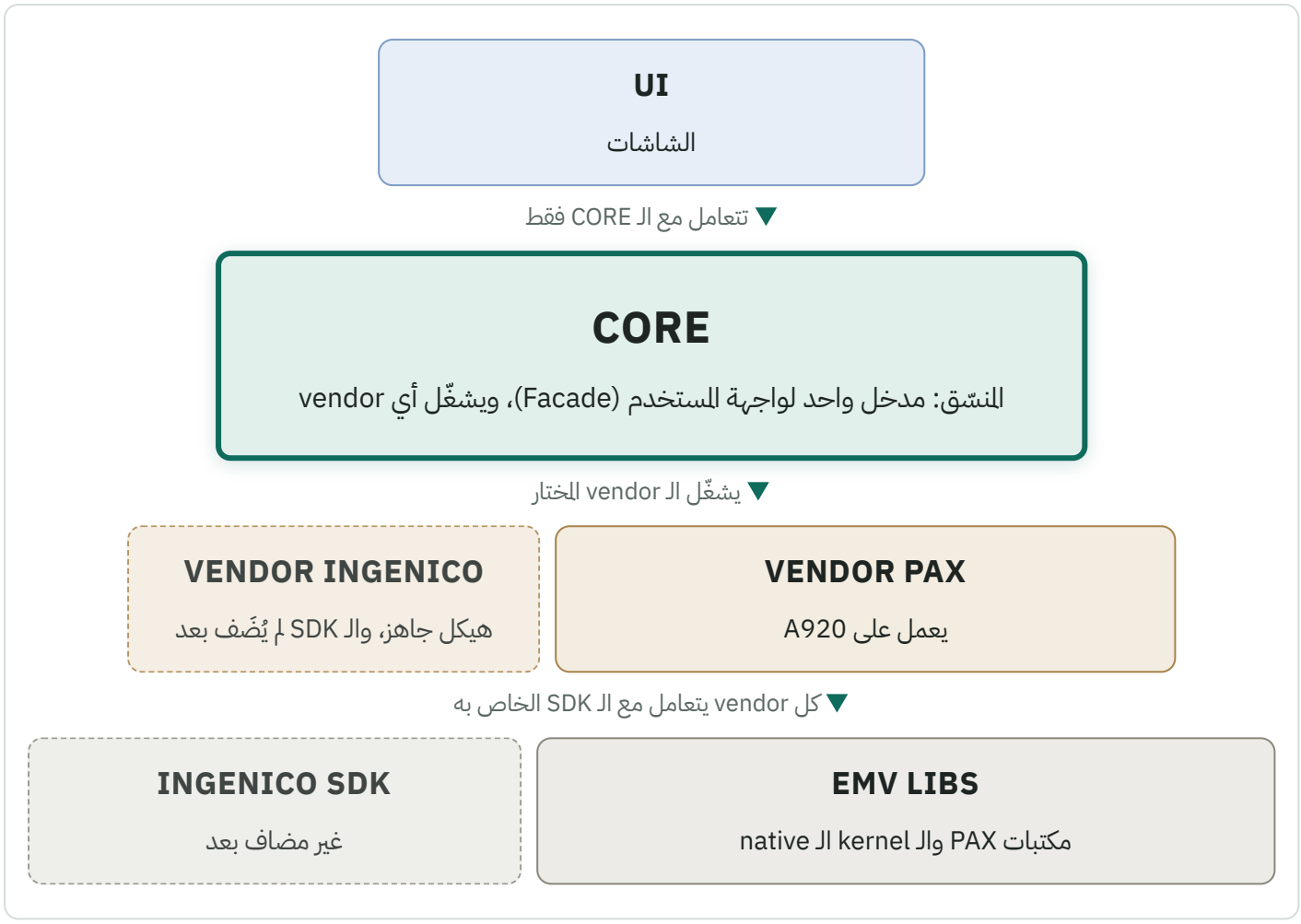
رحلة العملية من زر البيع حتى نهايتها: البطاقة، والـ kernel، والبنك، ثم التأكيد أو الإلغاء عند البنك. كل اسم في الصفحة موجود في الكود، وكل سهم استدعاء حقيقي. الشرح كله على فرع `transactionStepObserver`.

---

1. المكوّنات: من يملك ماذا
2. A sale from start to end
3. EMV steps: goToStep and onXxx
4. When the kernel waits: askStep
5. The way to the bank
6. The contact path
7. النتيجة والنهاية الآمنة
8. مسار الـ contactless
9. الـ EMV params
10. مبادئ البرمجة في المشروع
11. استخدامنا كـ SDK

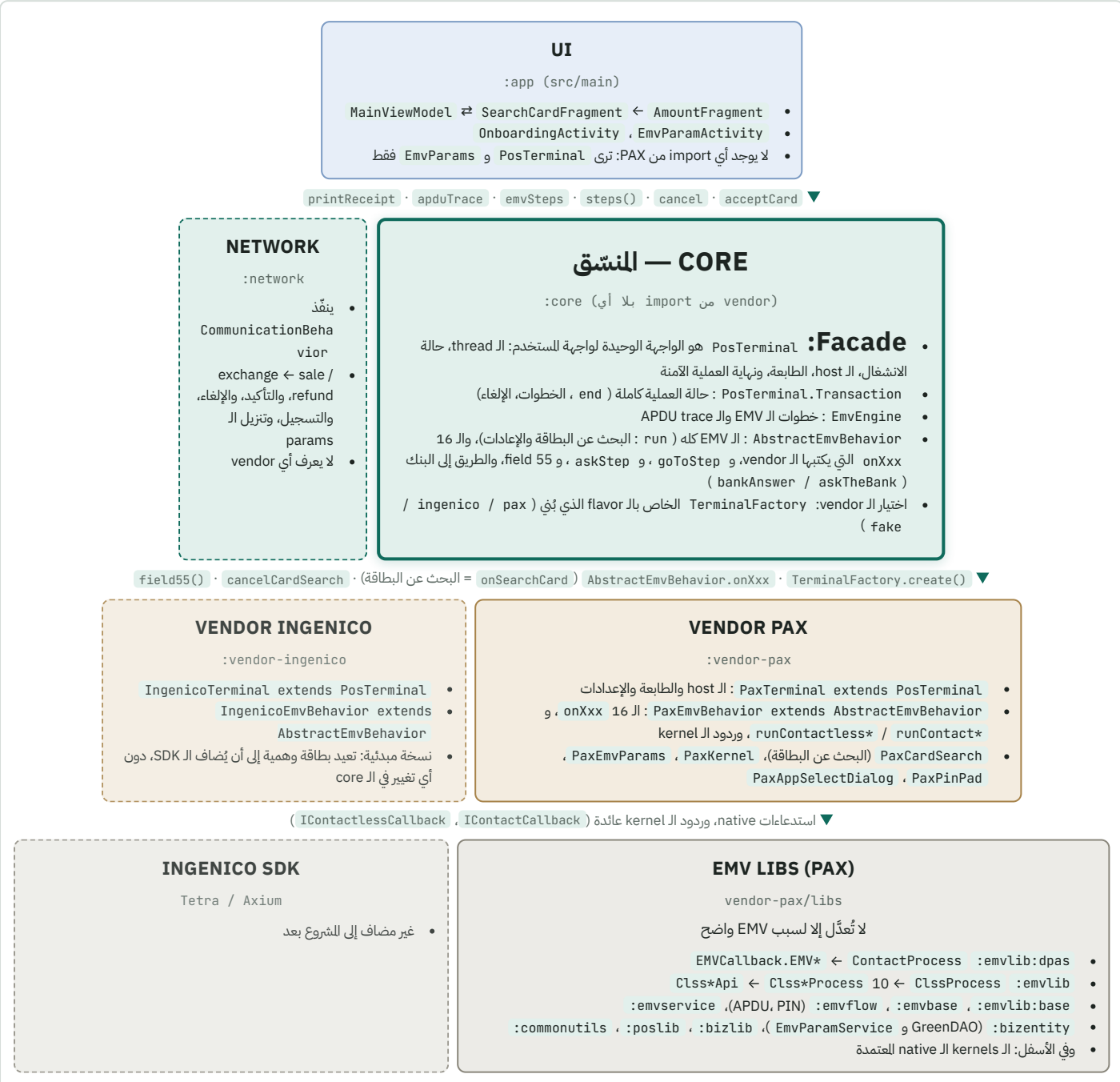
1. المكوّنات: من يملك ماذا

لكل مسؤولية مالك واحد. الشاشة لا تعرف PAX، والـ EMV لا يعرف البنك ولا الـ threads، و `PosTerminal` هو الباب الوحيد الذي تدخل منه الشاشة.


UI
الشاشات
▼ تتعامل مع الـ CORE فقط
CORE
المنسّق: مدخل واحد لواجهة المستخدم (Facade)، ويشغّل أي vendor
▼ يشغّل الـ vendor المختار
VENDOR INGENICO
هيكل جاهز، والـ SDK لم يُضَف بعد
VENDOR PAX
يعمل على A920
▼ كل vendor يتعامل مع الـ SDK الخاص به
INGENICO SDK
غير مضاف بعد
EMV LIBS
مكتبات PAX والـ kernel الـ native

الطبقات بالتفصيل

كل مربع هو module في الكود، وتحته أهم الكلاسات. ما بين الطبقتين هو الواجهة الوحيدة التي تمر من طبقة إلى التي تحتها.



- **الاستدعاءات تتجه إلى الأسفل فقط.** ما يصعد إلى الأعلى هو الخطوات (`Transaction.steps()` ، `emvSteps`)، وردود الـ kernel التي تعود إلى الـ vendor الخاص بها.
- **إضافة vendor جديد** تعني module جديدًا فيه `TerminalFactory` ، وكلاس يرث `PosTerminal` (الإعدادات)، وكلاس يرث `AbstractEmvBehavior` (الخطوات، ومنها البحث عن البطاقة). لا تتغير واجهة المستخدم ولا الـ CORE (`doc/VENDOR.md`).
- يوجد أيضًا `:vendor-fake` للاختبار، ويثبت أن الـ CORE يعمل دون أي SDK.

مكوّنات كل طبقة

كل بطاقة تمثل طبقة من الهرم، وبداخلها الـ packages، وكل كلاس ووظيفته في سطر.

UI `:app (src/main)`

الشاشات. تطلب من الـ CORE وتعرض ما يعود منه، ولا تحتفظ بأي حالة للعملية ولا تعرف أي vendor.

بدء التطبيق

`EmvEnhanceApp`

ينشئ `PosTerminal` واحدًا طوال عمر التطبيق (`TerminalFactory.create`)، ويحدد عنوان الـ host

عملية البيع

`MainActivity`

الشاشة الرئيسية: أزرار فقط

`AmountActivity` / `AmountFragment`

نوع العملية والمبلغ، ثم شاشة البطاقة

`SearchCardFragment`

شاشة العملية: الشريط العلوي، الإلغاء، والنتيجة

`MainViewModel`

يربط `PosTerminal` بالـ LiveData: `acceptCard` ، `cancel` ، الخطوات، الـ trace

`Event`

قيمة تُقرأ مرة واحدة (بسبب تدوير الشاشة)

الإعدادات

`EmvParamActivity` / `EmvParamListFragment`

الـ EMV params: من الـ cache أو من قاعدة البيانات، والتنزيل

`OnboardingActivity`

تسجيل الجهاز عند الـ host، و mToken، و PIN bypass

`CountryListActivity`

الدولة والعملة

CORE — المنسّق `:core`

العملية من بدايتها إلى نهايتها، بالكود نفسه لأي vendor: واجهة المستخدم تتعامل معه وحده، وهو يتعامل مع الـ vendor من خلال واجهات (abstract classes و interfaces).

`terminal` — الجهاز

`PosTerminal`

الـ Facade: المدخل الوحيد لواجهة المستخدم. thread واحد، حالة الانشغال، الذهاب إلى البنك (`sendToBank`)، الطابعة

`PosTerminal.Transaction`

عملية واحدة: حالتها (`end`)، وخطواتها (يشغّلها `TransactionStepObserver`)، والإلغاء، والطلبات المرسلة إلى البنك ونهايتها الآمنة

`EmvBehavior`

الواجهة: `run` · `cancelCardSearch` · `dispose`

`BankRequest`

طريق الـ EMV إلى البنك: `send(config)` ؛ يعطيه `PosTerminal` للـ behavior بـ `attach(this::sendToBank)`

`AbstractEmvBehavior`

الـ EMV المشترك: `run` (المحاولات والبحث والإعادة: `requestRetry` / `retryCount()`، `goToStep` ، `askStep` ، 55 field، والـ 16 `onXxx` التي يكتبها الـ vendor. موجود هنا لأنه يستدعي `Transaction` باستدعاءات package-private

`TerminalBusyException`

محاولة بدء عملية بينما أخرى قيد التنفيذ

`transaction` — العملية

`TransactionConfig`

النوع، المبلغ، طريقة إدخال البطاقة، field 55، الـ PIN block، `asyncRequestId`

`TransactionType`

sale / refund / cash in

`TransactionStep / Event`

مراحل العملية: البدء، انتظار البطاقة، الاتصال بالـ host، الموافقة، التأكيد أو الإلغاء... والخطوات التي تقود العملية (البدء، الإلغاء، النتيجة، التأكيد / الإلغاء عند الـ host، النهاية) لكل منها دالة

`emv` — الـ EMV

`EmvEngine`

يرسل كل خطوة EMV إلى من يشغّلها وإلى الشريط العلوي، ويحتفظ بالـ APDU trace. لا علاقة له بحالة العملية ولا بإعادة المحاولة ولا بالبنك

`EmvStep / Event`

الخطوات الـ 17 الخاصة بالـ kernel (EMV Book 3)

`EmvTransactionResult`

ما قُرئ من البطاقة ويحتاجه الـ host

`Field55`

يبني 55 field من الـ tags بالطريقة نفسها لكل vendor

`ApduTrace`

سجل الـ APDU والخطوات (في نسخة الـ debug فقط)

`card` — البطاقة

`CardPresence / CardSearchListener`

البطاقة التي عُثر عليها (`isEmv()` / `isNonEmv()`)، وأحداث البحث

`EntryMethod`

chip / tap / swipe / manual

`host` — الواجهات الخارجية

`CommunicationBehavior`

الـ host: `connectToHost` · `reverse` · `acknowledge`

`AuthResult`

رد الـ host: approved / declined / failed / notSent

`PrinterBehavior`

الطابعة

`HostDefaults`

host وطابعة وهميان للعرض التجريبي

`cardholder` — انتظار حامل البطاقة

`AbstractAppSelectDialog`

اختيار التطبيق على البطاقة: الانتظار والمهلة، ولا يختار الأول تلقائيًا أبدًا

`PinPadBehavior / AbstractPinPadBehavior / PinKey`

شاشة الـ PIN: النقاط والأزرار فقط، والتشفير من مهام الـ SDK

`util · param`

`EmvParams`

واجهة شاشة الـ EMV params

`EmvLog`

السجلات

NETWORK `:network`

التواصل مع الـ host الخاص بنا. ينفّذ `CommunicationBehavior` ولا يعرف أي vendor.

عملية البيع

`SaleCommunicationBehavior`

sale / refund ← exchange، والتأكيد والإلغاء، ويحدد لكل فشل: هل يمكن أن يكون الطلب قد وصل إلى الـ host أم لا

`HostApiConnection` / `HostApiClient`

الـ endpoints (Retrofit) والـ client

`HostHeaders` / `HostSettings` / `HostAppKeys`

الـ headers: الـ token، و mToken (يُعدَّل من الجهاز)، والمفاتيح الثابتة

`PosEntryModeCodes`

051 / 071 / 901 / 011

`tlv` · `crypto`

`RsaPublicKeyEncryptor`

يشفّر مفتاح العملية (TEK) بمفتاح الـ host

`AesEnvelopeCrypto`

يشفّر الطلب والرد ويفك تشفيرهما

`BerTlv` / `IssuerResponseFields`

يقرأ الـ ARPC والـ scripts من رد الـ host

`onboarding` · **params**

`OnboardingClient` / `OnboardingState`

تسجيل الجهاز، والمفاتيح، والـ token وتجديده

`EmvParamDownloadClient`

ينزّل ملف الـ params (البيانات فقط، والقراءة لدى PAX)

`EnvironmentProvider`

عنوان الـ host الخاص بالـ build

`PinBypassSettings`

هل الـ PIN bypass مسموح أم لا

VENDOR PAX `:vendor-pax`

PAX A920: ينفّذ واجهات الـ CORE باستخدام SDK الخاص بـ PAX. هذا هو المكان الوحيد الذي يظهر فيه اسم PAX.

البدء

`TerminalFactory`

المكان الوحيد الذي يُنشئ `PaxTerminal`

`VendorApplication`

الـ Application الخاص بـ PAX (الـ SDK يحتاج `BaseApplication`)

`PaxHardwarePermissions`

صلاحيات قارئات ICC / Mag / PICC

`PaxSetup`

الدولة والعملة، والبيانات الأولية عند أول تشغيل (خرجت من `PaxTerminal`)

البطاقة والـ EMV

`PaxTerminal`

`extends PosTerminal` : الـ host والطابعة الفعليان، والإعدادات

`PaxCardSearch`

البحث عن البطاقة (polling): عمل الخطوة `SEARCH_CARD` (`onSearchCard`)، و `stop()` عند الإلغاء

`PaxEmvBehavior`

`extends AbstractEmvBehavior`: الـ 16 `onXxx` (استدعاء native لكل خطوة)، والرد على أسئلة الـ kernel، و `field55()`

`PaxKernelSession`

ما عُرف خلال المحاولة الحالية، ويُنشأ من جديد مع كل محاولة

`PaxKernel`

كائنات الـ kernel: params، contact، contactless، mag

مساعدات الـ EMV

`PaxPinEntry`

الـ PIN: اللوحة، مفتاح الـ online PIN (AES) ولفّه بـ RSA للـ host، وقواعد الـ bypass. مفتاح الـ host العام يأتيه من `PaxTerminal` ، فلا يقرأ الـ EMV حالة الـ host بنفسه

`PaxHostAnswer`

رد الـ host بالشكل الذي يأخذه الـ kernel للـ 2nd GENERATE AC (`toOnlineResult`)

`PaxReaders`

إغلاق القارئات (mag، ICC، PICC) كلٌّ على حدة

`PaxCardData`

بيانات البطاقة كنص: الـ PAN من track 2 أو tag 5A، وإخفاء الـ PAN، والـ tags النصية (دوال ثابتة، وعليها أول اختبارات PAX)

الشاشات والأجهزة

`PaxPinPad`

شاشة الـ PIN

`PaxAppSelectDialog`

اختيار التطبيق

`PaxPrinter`

الطابعة الحرارية

`PaxEmvParams`

بيانات شاشة EMV Params (الـ cache / قاعدة البيانات / التنزيل)

VENDOR INGENICO `:vendor-ingenico`

الواجهات نفسها تمامًا، لكن بنسخة مبدئية: تعيد بطاقة ونتيجة وهميتين إلى أن يُضاف الـ SDK. عند إضافته لن يتغير الـ CORE ولا واجهة المستخدم.

`TerminalFactory` / `VendorApplication`
البدء

`IngenicoTerminal`
`extends PosTerminal` : الـ host والطابعة الوهميان

`IngenicoEmvBehavior`
`extends AbstractEmvBehavior`: بحث وهمي (`onSearchCard` ؛ لاحقًا Tetra / Axium)، وكل خطوة تنتقل إلى التالية

EMV LIBS (PAX)

`vendor-pax/libs/sdk — emvlib · emvbase · emvservice · bizlib · poslib · commonutils · vendor-pax/libs/ours — emvflow · bizentity`

الـ SDK الخاص بـ PAX والـ kernels الـ native المعتمدة، وكلها داخل الـ vendor في `vendor-pax/libs/` : المجلد `sdk/` هو كود PAX نفسه، و `ours/` هو كودنا (`emvflow` و `bizentity`). لا يتعامل معها إلا vendor-pax (مسارات Gradle مثل `:vendor-pax:libs:sdk:poslib` ، وتُكتب هنا مختصرة `:poslib`)، ولا تُعدَّل إلا لسبب EMV واضح.

الـ kernels

`:emvlib:dpas — ContactProcess`
الـ contact kernel: من `EMVAppSelect` إلى `EMVCompleteTrans` ، و `EmvCallBackListener`

`:emvlib — ClssProcess + Clss*Process`
الـ contactless kernel: لكل scheme (PayPass، PayWave، ...)

`:emvlib:base` / `:emvbase`
الكلاسات الأساسية، والـ params، والنتائج، والردود

الجهاز

`:emvflow`
EmvFlowRuntime (DAL)، EmvDeviceImpl (APDU، PIN، trace)

`:emvservice`
PinService، MagCardService

`:poslib` / `:commonutils` / `:bizlib`
أدوات PAX الخاصة بالجهاز

الـ params

`:bizentity`
GreenDAO (قاعدة البيانات)، `EmvParamService` (الـ cache)، قارئات الـ XML، `EmvParamUpdater` ، `ConfigInit` (أول تشغيل)

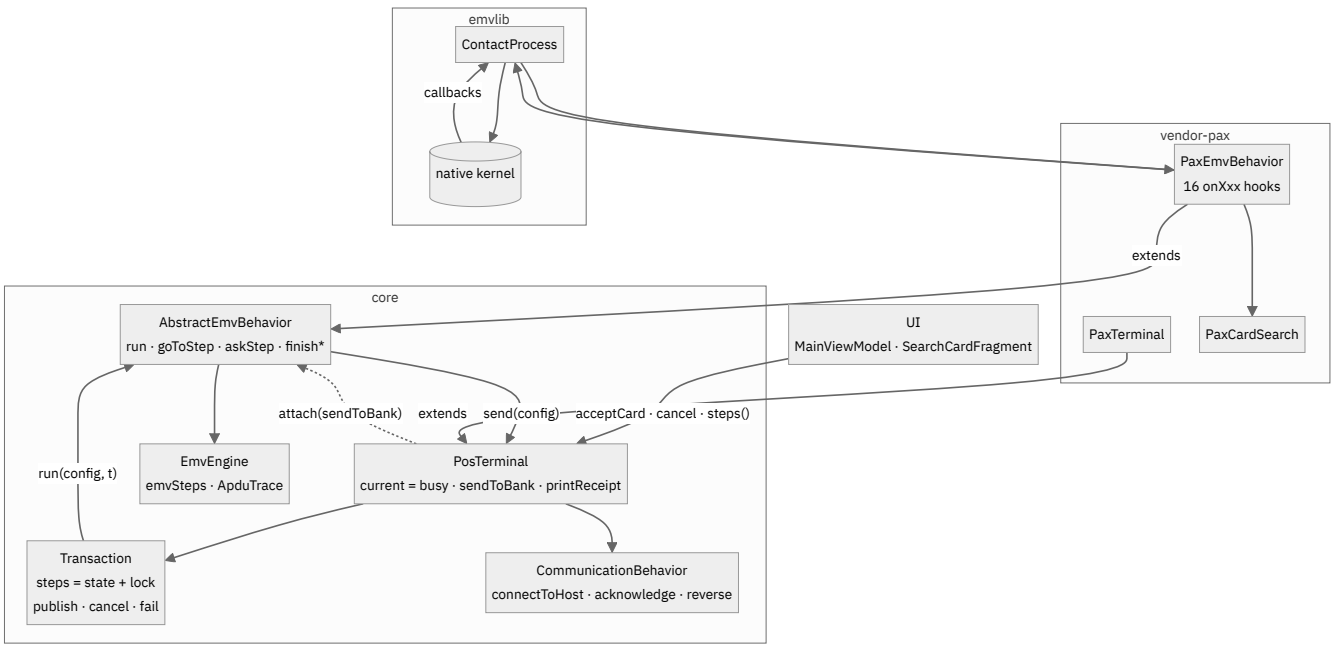
`PosTerminal` is the machine

```
// PosTerminal = the POS machine
AbstractEmvBehavior   behavior;       // the card: search, reader, kernel
CommunicationBehavior communication;  // HTTP to the bank
PrinterBehavior       printer;        // the receipt
Scheduler             transactionScheduler;
                      // one sale at a time, on one thread
```

Class	Owns	Must not
`PosTerminal`	One sale at a time (`current` = busy), the call to the bank (`sendToBank`), the printer.	Hold any EMV logic.
`PosTerminal.Transaction`	The sale's state, read from its steps (`decision()`); the steps themselves; cancel; the requests sent to the bank and their safe closing.	Publish anything after its end.
`AbstractEmvBehavior`	The shared EMV: attempts, `goToStep` , `askStep` , `askTheBank` / `bankAnswer` , and the 16 `onXxx` hooks a vendor writes.	Know about threads, the busy state, or call the bank directly.
`PaxEmvBehavior`	The `onXxx` hooks on PAX: one native call per step, and the kernel callbacks.	Be called directly (only through `goToStep`).
`EmvEngine`	The EMV step stream (`emvSteps` : one for the terminal, never completes) and the APDU trace.	Hold any transaction state.
`SaleCommunicationBehavior`	The bank: `connectToHost` (token → exchange → sale / refund), acknowledge, reverse. Decides whether a request reached the bank.	Decide to acknowledge or reverse.

Who calls whom

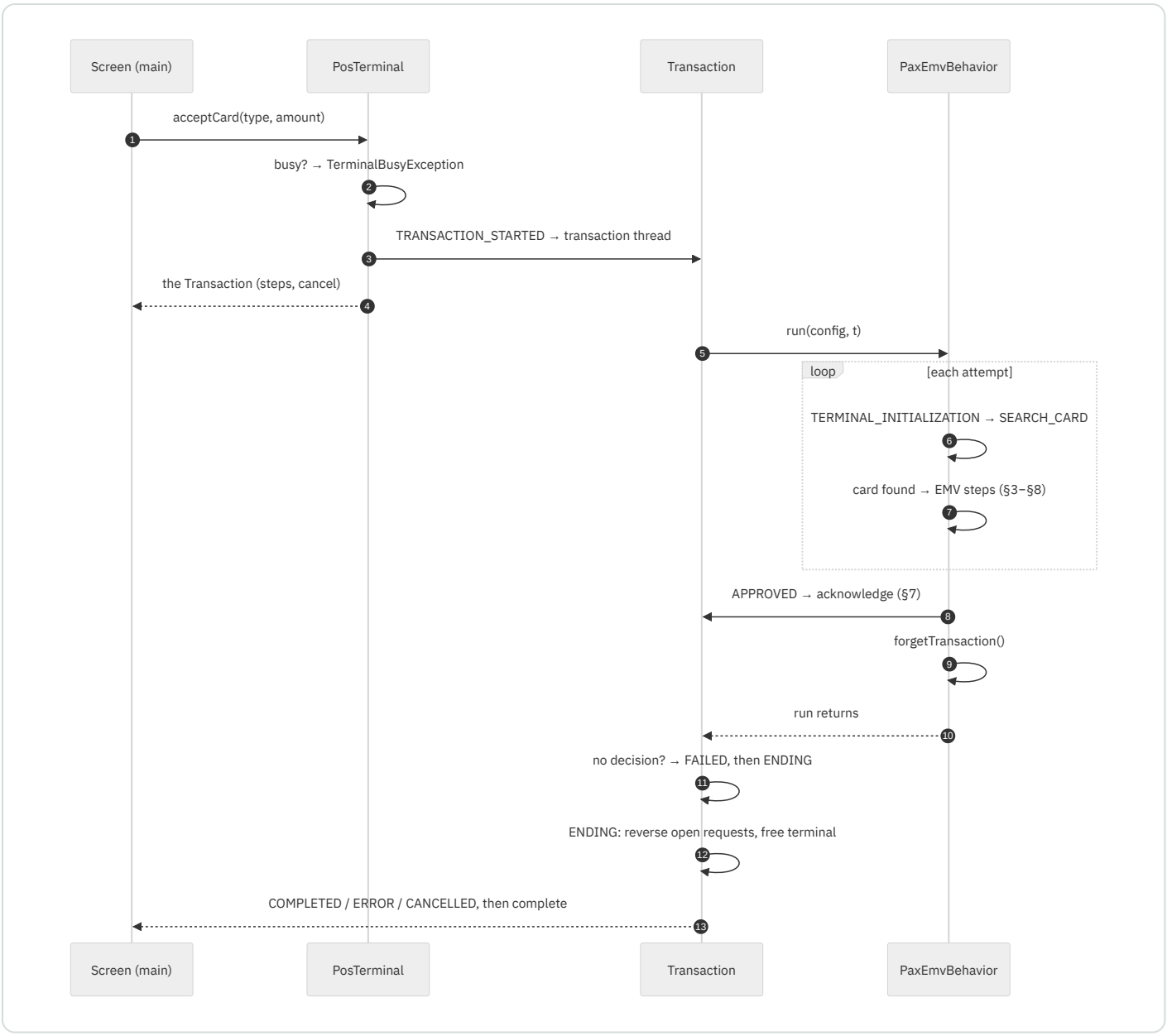
A solid arrow is a real call. A dashed arrow is wired once, at start-up.



2. A sale from start to end

A sale runs on its own steps (`TransactionStep`). Every step enters through one method, `publish` , and is stored in `Transaction.steps` . `TransactionStepObserver` then runs the step's case in `dispatchTransactionStep` . A step with no case is only a report for the screen.

```
// TransactionStepObserver.onNext → dispatchTransactionStep(event)
switch (event.getStep()) {
  case TRANSACTION_STARTED: onTransactionStarted(event);
      // move to the transaction thread
  case CANCELLING: onCancelling();
      // stop the card search
  case APPROVED:   closeLast(ACKNOWLEDGING);
      // bank and card approved → acknowledge
  case DECLINED:   closeLast(REVERSING);
      // bank approved, card declined → reverse
  case ENDING:     onEnding();
      // reverse what is still open, free the terminal, deliver the end
  default:
      // a report: ONLINE_*, CARD_*, FAILED, the closings, the end
}
```

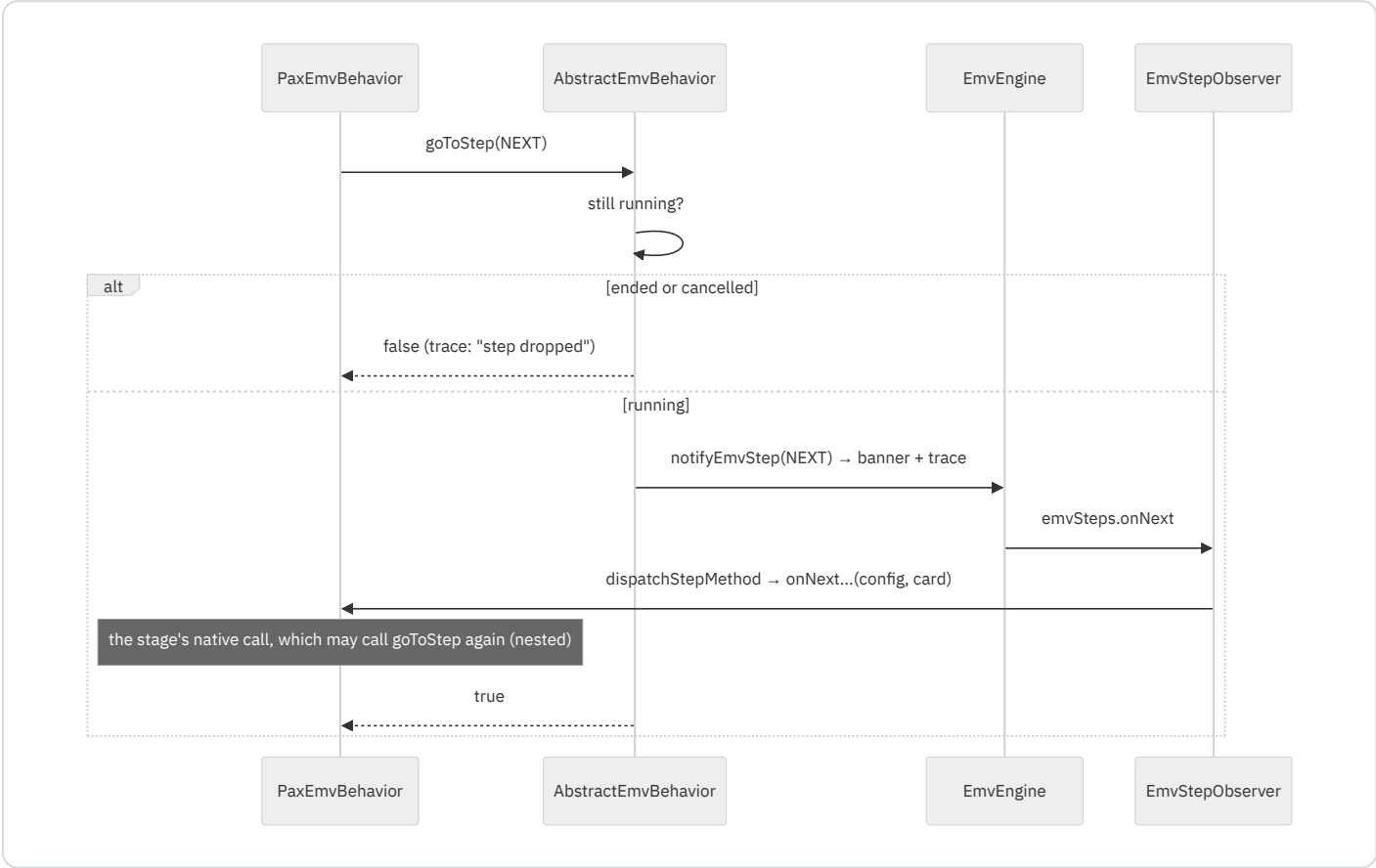


- **Busy.** Checked on the caller's thread. A second sale is refused at once with `TerminalBusyException`; nothing is queued.
- **The state is the steps.** There is no state field. The first decision step delivered is the state (`decision()`): `CANCELLING`, `APPROVED`, `DECLINED` or `FAILED`. No decision means the sale is running. The first decision wins; later ones are dropped.
- **The end follows the decision.** `CANCELLING` → `CANCELLED`; `APPROVED` / `DECLINED` → `COMPLETED`; `FAILED` → `ERROR` with the same message. `FAILED` exists because `ERROR` must stay the last step.
- **Cancel.** `cancel()` publishes `CANCELLING`. The card search stops, and the EMV stops at its next `goToStep`. The thread is never interrupted.
- **Retry** (fallback, tap again, See Phone). The EMV calls `requestRetry(...)` and the kernel returns. `run` then starts a new attempt from `TERMINAL_INITIALIZATION`, in the same sale.

- **The result is not the end.** `APPROVED` / `DECLINED` / `FAILED` are published as soon as they are known. `COMPLETED` comes later, after the bank is acknowledged or reversed and the terminal is free.
- **Nothing stays attached after the end** (`213762c`):
  - The steps complete after the end (`steps.onComplete()`). Every subscriber is released without having to dispose. The steps stay readable, so a late subscriber still receives all of them.
  - `run` ends with `forgetTransaction()` . The EMV dispatcher is disposed, and the sale's data is cleared: the config (with field 55), the card, the bank answer.
  - `emvSteps` never completes. It belongs to the terminal, not to the sale; completing it would block the next sale. The screen listens to it per sale, until the sale ends (`takeUntil`).
- A typical run on screen: `ONLINE_REQUIRED → ONLINE_PROCESSING → ONLINE_COMPLETED → APPROVED → ACKNOWLEDGING → CLOSING_COMPLETED → ENDING → COMPLETED` .
- ⚠ The step cases run while the steps lock is held, including the bank calls of the closings. `publish` therefore checks the decision before taking the lock, so `cancel()` never waits for a closing. Steps that may wait for the bank or a person (`ONLINE_PROCESSING` , `SINGLE_TAP_PIN_REQUIRED`) stay reports, with no case.

3. EMV steps: how `goToStep` runs `onXxx`

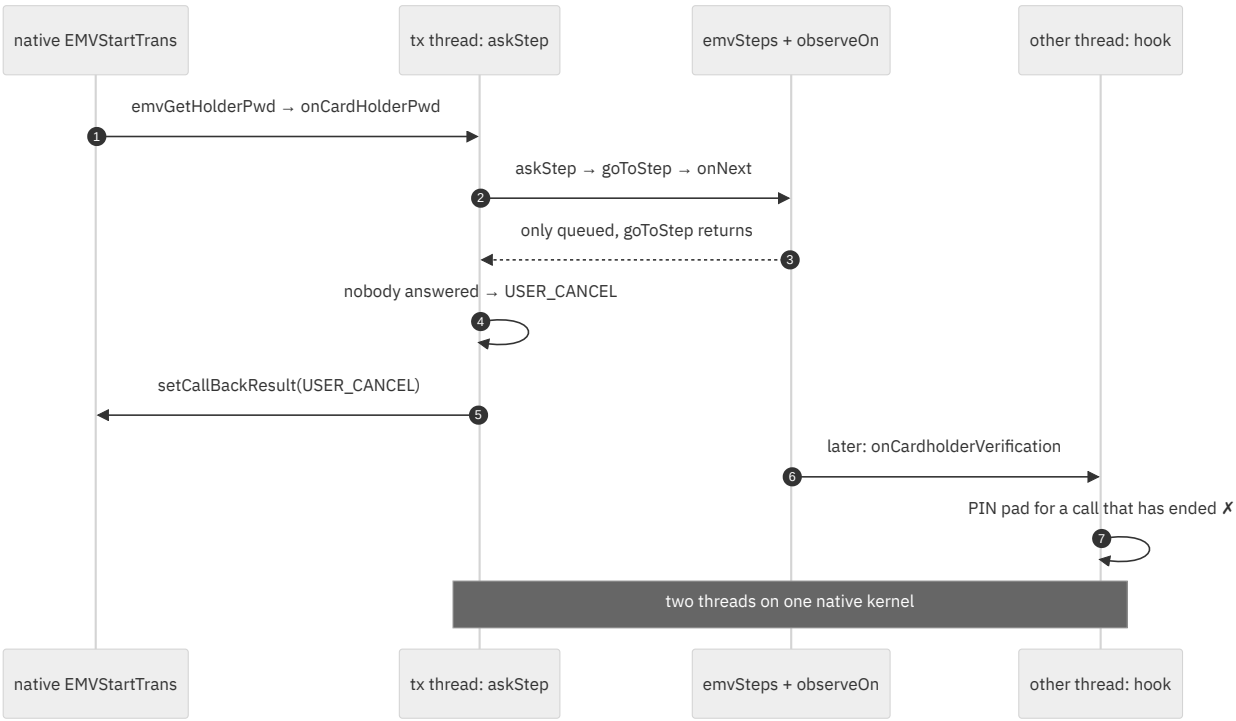
Nobody calls an `onXxx` hook directly. `goToStep(STEP)` publishes the step on `emvSteps`. `EmvStepObserver` runs its hook on the same thread, and the hook finishes before `goToStep` returns. If the sale has ended or was cancelled, the step is dropped and `goToStep` returns `false`.



- **Start.** `TERMINAL_INITIALIZATION` prepares the kernel, then calls `searchCard()`, which starts `SEARCH_CARD`. If preparation fails, the sale ends with `ERROR` and no search runs.
- **The dispatch must stay synchronous:** no `observeOn` and no serialize on `EmvStepObserver`. The table below explains why.
- **The screen is the exception.** It may use `observeOn(main)` on `emvSteps`, because it only shows the banner and nothing waits for it. It subscribes per sale, so it can miss the first EMV step; the dispatch is not affected.

Why the dispatch must stay synchronous

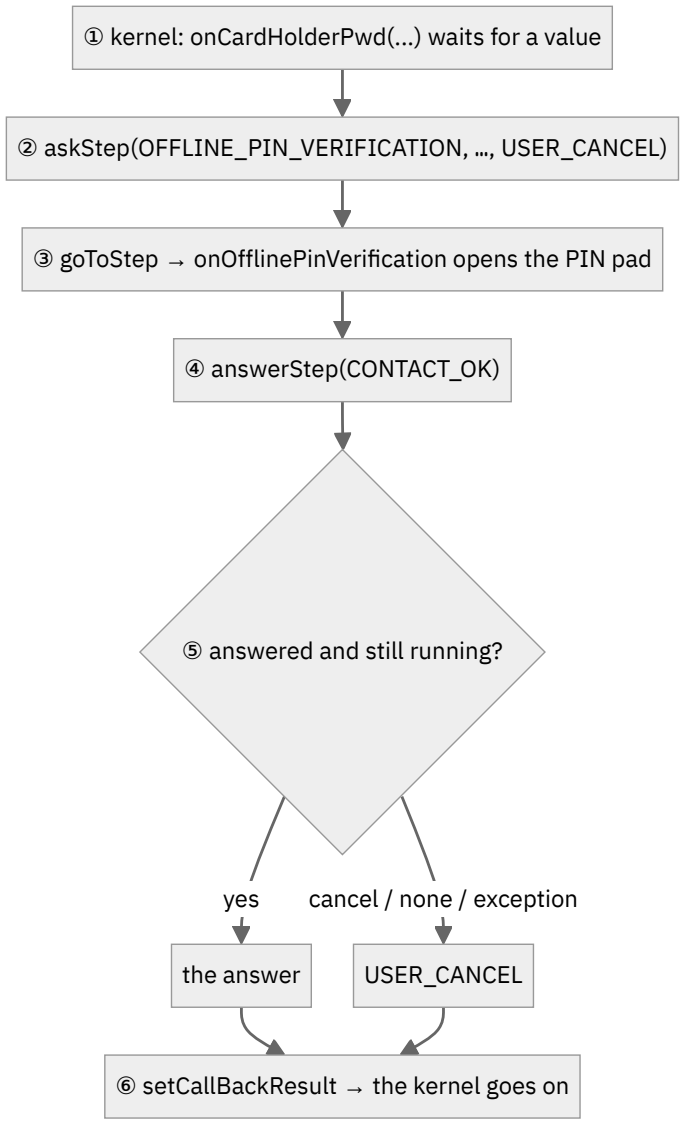
`observeOn(X)` queues `onNext` and runs it later on another thread, so `goToStep` returns at once. On `EmvStepObserver` that breaks the flow:



If we use	What happens	Why
`observeOn` on the observer	`askStep` answers `USER_CANCEL` before the PIN pad or the app-selection dialog opens.	The hook runs after `goToStep` has returned, when nobody has answered yet.
`observeOn` on the observer	Two threads call one kernel: a hook calls a native function (`EMVReadAppData` , `EMVCardAuth`) while the transaction thread is still inside another, or `finishContactStage` closes the readers under it.	The PAX kernel and readers are not thread-safe. Everything must run on one thread, in order.
`observeOn` on the observer	The order breaks: `runContactAppSelect` continues as if the read stage had finished and stored its result in `tx.transResult` .	Each stage reads the previous stage's result after `goToStep` returns.
`emvSteps.toSerialized()`	A nested `goToStep(NEXT)` is queued until the current `onNext` returns.	Every hook calls `goToStep` from inside `onNext` . `askStep` then returns its fail-closed value, and the stage finds its result missing.

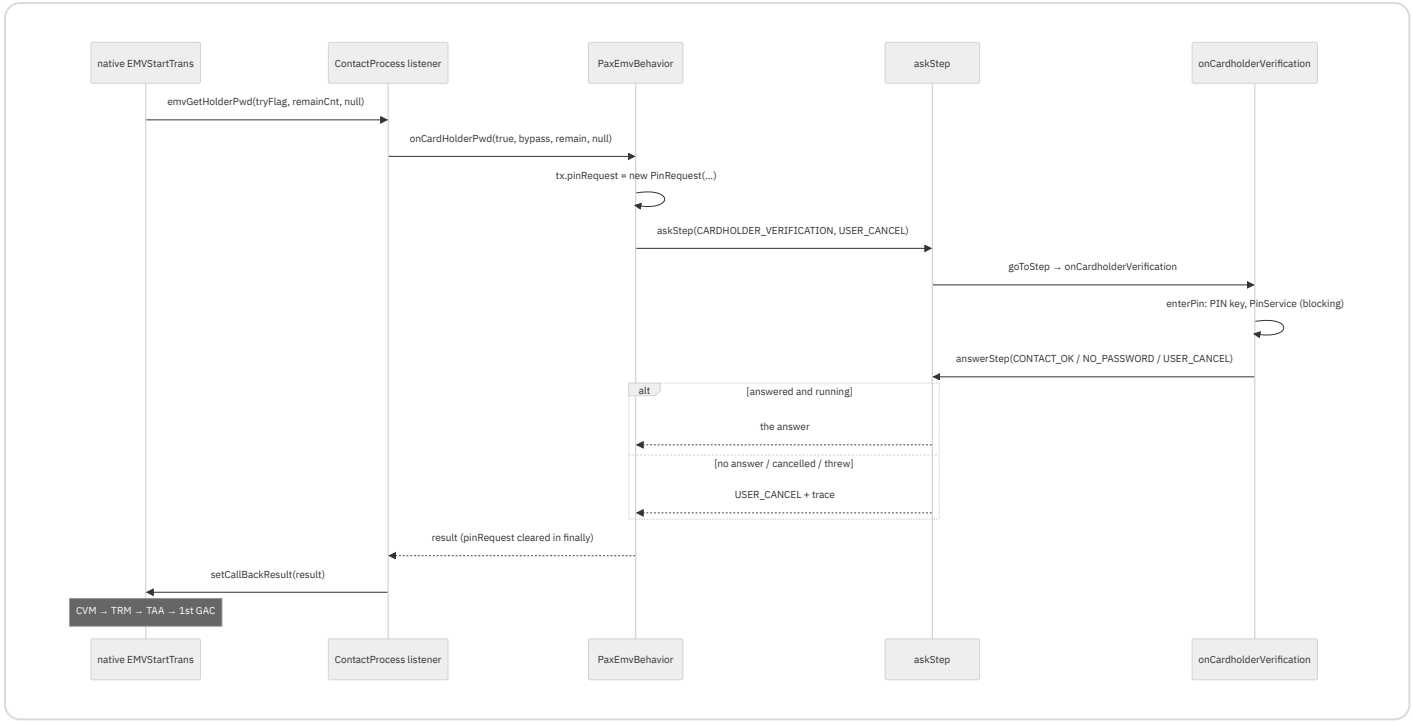
4. When the kernel waits for a person: `askStep`

During a native call (for example `EMVStartTrans`) the kernel stops and asks through a callback: choose the application, enter the PIN. It must always get an answer (`setCallBackResult`), or it waits forever. The callback does not open a screen itself; it asks the responsible step through `askStep` .



- **Fail closed.** On cancel, no answer or an exception, `askStep` answers `USER_CANCEL` at once. The kernel never waits, and never approves without an answer.
- Callbacks that ask a person: `onWaitAppSelect` (application selection) and `onCardHolderPwd` (PIN). Data callbacks (`emvSetParam`, `emvInputAmount`) answer inside `ContactProcess`, with no step.

In detail: online PIN

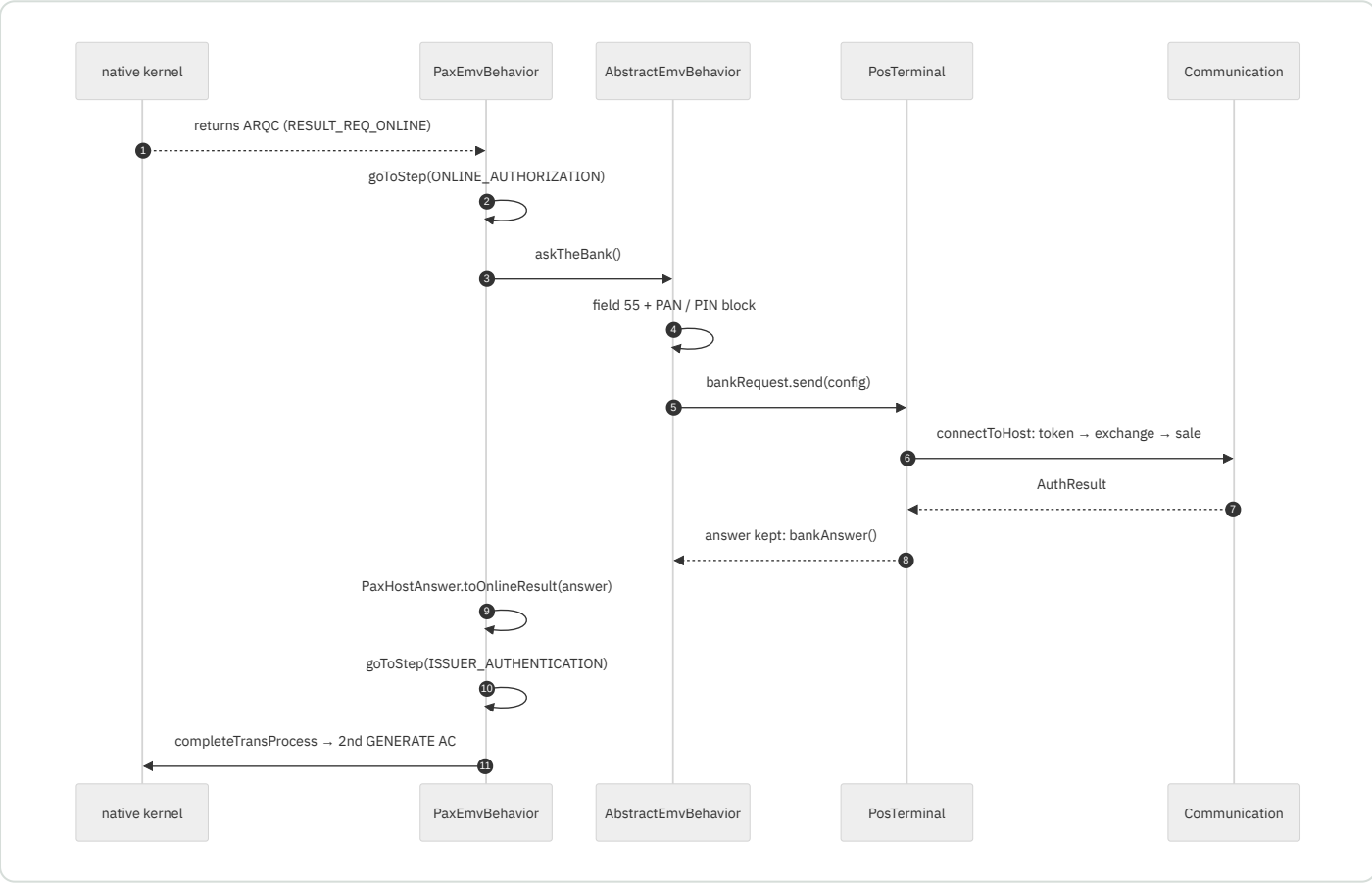


- `askStep` works only because the hook finishes before `goToStep` returns (§3). `answerStep` without `askStep` does nothing.
- **Offline PIN.** `CARDHOLDER_VERIFICATION` is published as an announcement, then `askStep(OFFLINE_PIN_VERIFICATION)`. Ours is the screen only; the kernel checks the PIN with the card.
- **Application selection.** `emvWaitAppSel` → `onWaitAppSelect` → `askStep(WAIT_ / FINAL_APPLICATION_SELECTION)` → `answerApplicationSelection` (a dialog, or the only application when there is one).
- Data callbacks stay in `ContactProcess` and ask no step: `emvInputAmount`, `emvSetParam` (→ `resetParam(AID)`), `emvUnknowTLVData`. `certVerify` and `emvVerifyPINfailed` answer with `setCallBackResult`.

5. The way to the bank

When the card answers the 1st GENERATE AC with an ARQC, the native call **returns** with `RESULT_REQ_ONLINE`. Our code then calls the bank, and goes back to the kernel for the 2nd GENERATE AC in a later step. Three layers take part:

Layer	Method	Does
EMV	`askTheBank()`, in step `ONLINE_AUTHORIZATION`	Builds field 55 and the card data, asks the bank, keeps the answer (`bankAnswer()`).
Terminal	`PosTerminal.sendToBank(config)`	Names the request (`asyncRequestId`), records it for the safe end, publishes `ONLINE_*`.
Bank	`CommunicationBehavior.connectToHost`	token → exchange → sale / refund, over HTTP.



What the screen sees after the bank

Case	Steps after `ONLINE_COMPLETED`
Bank approved, card approved (TC)	`APPROVED → ACKNOWLEDGING → CLOSING_COMPLETED → ENDING → COMPLETED`
Bank approved, card declined (AAC)	`DECLINED → REVERSING → CLOSING_COMPLETED → ENDING → COMPLETED`
Bank declined	`DECLINED → ENDING → COMPLETED`
No usable answer (`failed`)	`FAILED → ENDING → REVERSING → CLOSING_COMPLETED → ERROR`
Never sent (`notSent`)	`FAILED → ENDING → ERROR`
Bank asks for the PIN (Single Tap, §8)	`SINGLE_TAP_PIN_REQUIRED` → the PIN → `ONLINE_*` again → one of the rows above

- **Two copies of the answer, each with one owner.** `bankAnswer()` is the EMV's: it carries on with the card, and is cleared at each attempt. `OnlineRequest.answer` is the terminal's: it decides the acknowledgement or the reversal. Neither reads the other's copy.
- `askTheBank` **always keeps its answer,** even when it returns early (no field 55, or a cancel), so the vendor reads the real reason.
- **Swipe and manual entry** have no kernel: `onReadApplicationData` goes straight to `ONLINE_AUTHORIZATION`, and the result comes from `bankAnswer()` (`finishNonEmv`).
- `startOnlineProcess()` **is never called.** It is part of PAX's callback interfaces, but neither the kernel nor our code calls it; going online is the `ONLINE_AUTHORIZATION` step. It stays only because the interface requires it.

How the EMV reaches the bank: `attach`

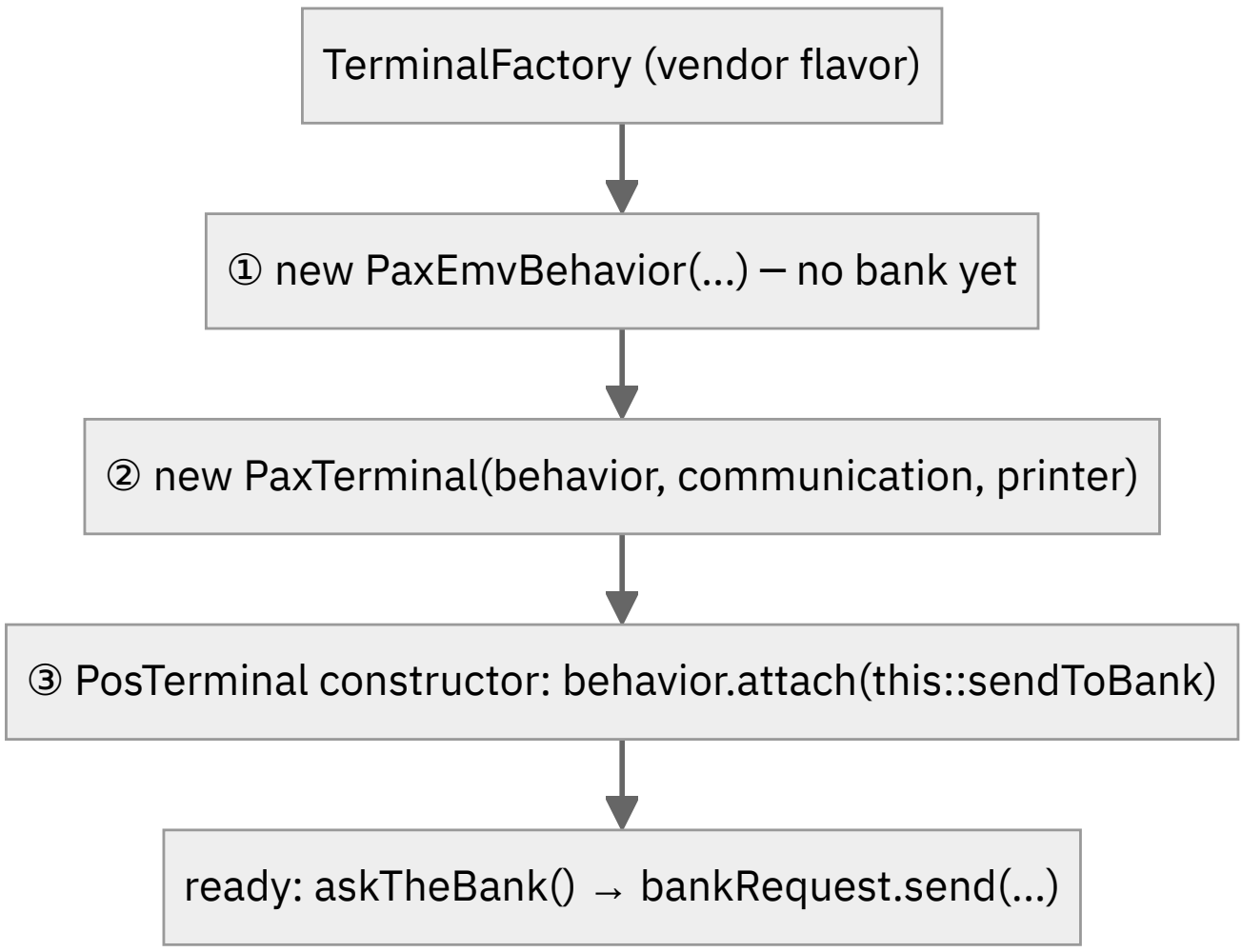
The EMV must call the bank, but must not know `PosTerminal`. So `PosTerminal` gives it one thing to call, a `BankRequest`, once, when it is created:

```
// BankRequest.java
public interface BankRequest { AuthResult send(TransactionConfig config); }

// AbstractEmvBehavior
private BankRequest bankRequest;
public final void attach(BankRequest bankRequest) {
    this.bankRequest = bankRequest;
}
protected AuthResult askTheBank() {
    … return bankAnswer = bankRequest.send(config);
}

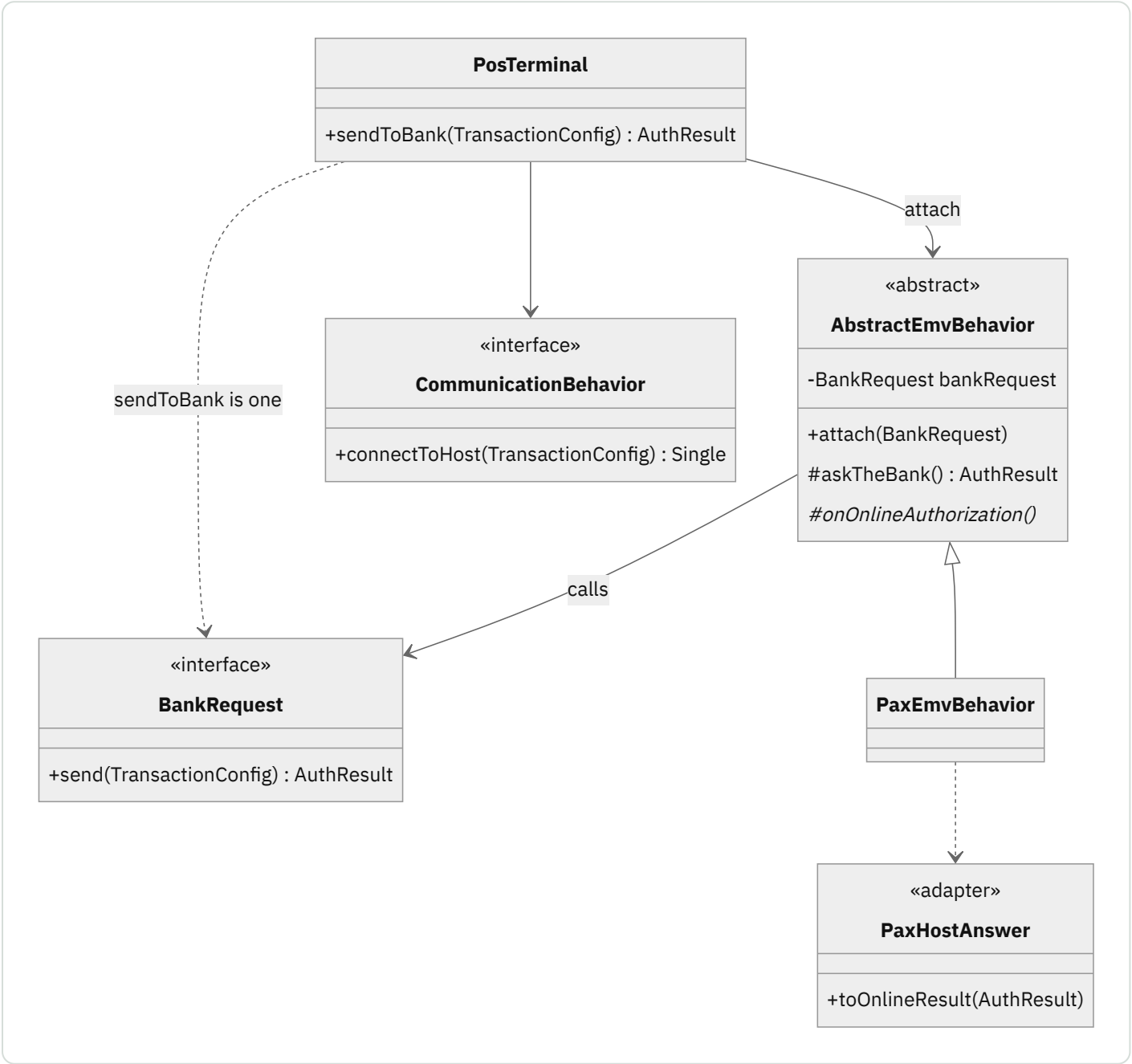
// PosTerminal constructor, once
behavior.attach(this::sendToBank);
```

Why not pass it in the constructor? The two objects need each other. `PosTerminal` needs the behavior in its constructor, and the behavior needs `PosTerminal::sendToBank`, which exists only after `PosTerminal` is built. So the behavior is built first, and wired to the bank right after:



Option	Result
The behavior's constructor takes a `BankRequest`	Not possible: `PosTerminal` does not exist yet.
`PosTerminal` creates the behavior	Rejected: the core would have to know vendor classes.
Dagger / Hilt	Same cycle; its fix (`Lazy` , `Provider` , injection after creation) is `attach` behind annotations. Nothing gets simpler.
`attach` **, once**	**Chosen.** The field is `private` , `attach` is `final` , and it runs before any sale.

The design behind it



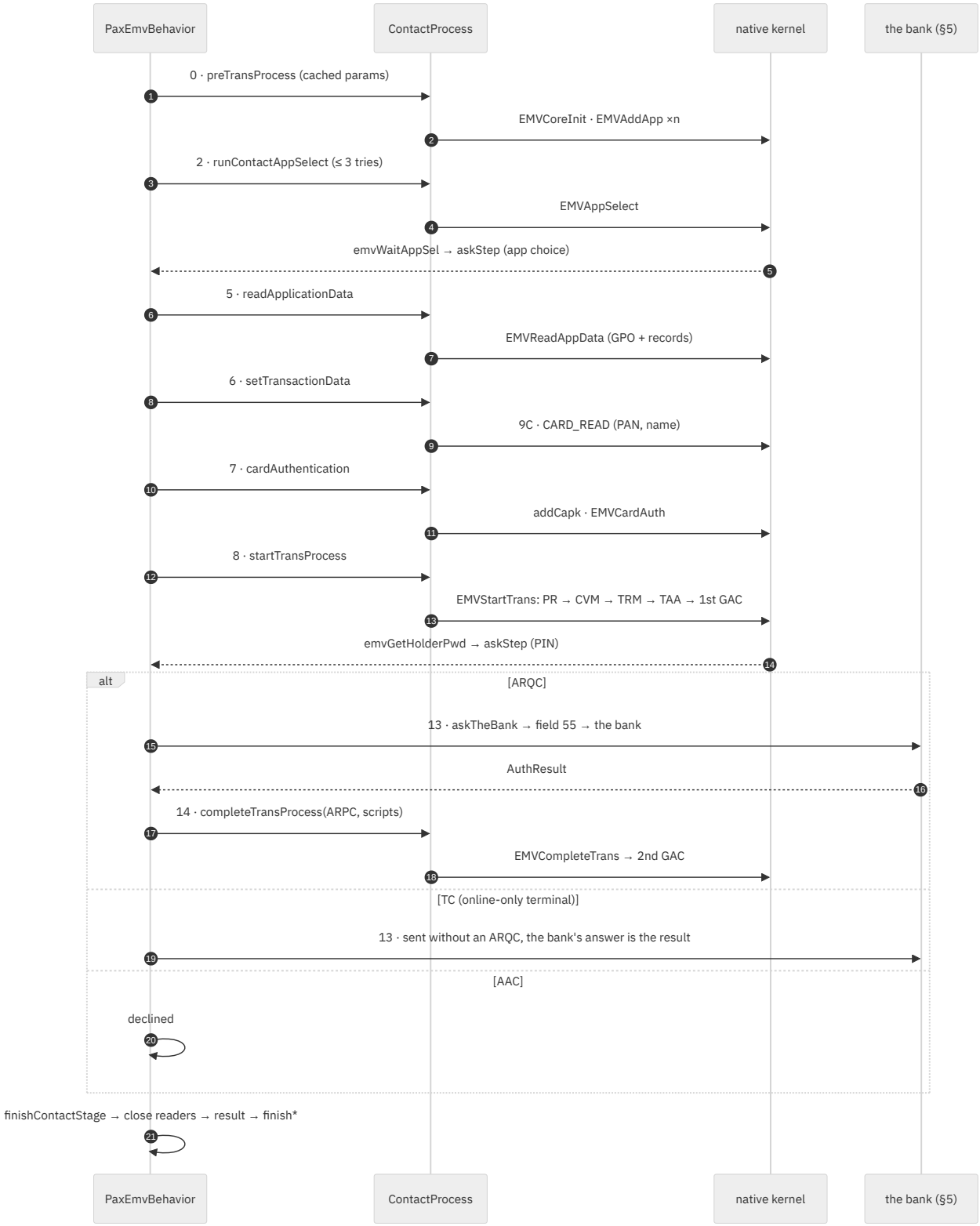
Concept	Where	What it gives us
Dependency Injection (no framework)	Each class gets what it needs from outside: `PosTerminal` its behavior, communication and printer in the constructor; the behavior its `BankRequest` through `attach`.	Each class keeps one job (Single Responsibility).
Dependency Inversion	The behavior depends on the `BankRequest` interface, not on `PosTerminal`.	The EMV knows nothing of the terminal or the bank. A test passes a one-line fake.
Callback via a method reference	`this::sendToBank` is the `BankRequest`.	No extra class implements the interface.
Template Method	`askTheBank()` is shared; `onOnlineAuthorization()` is each vendor's.	Checks, field 55 and the call live in one place; the vendor decides what to do with the answer.
Polymorphism	`CommunicationBehavior`, `AbstractEmvBehavior` (PAX, Ingenico, Fake).	The bank or the vendor changes without touching `PosTerminal`.
Value Object	`AuthResult`: private constructor, factories (`approved`, `declined`, `failed`, `notSent`, `pinRequired`), no setters.	The answer never changes after it is made, and has one vendor-free shape.
Adapter	`PaxHostAnswer.toOnlineResult(AuthResult)`	Turns the shared answer into the shape the PAX kernel reads. Another vendor writes its own.

> All of this is synchronous on the transaction thread: `askTheBank` waits for `sendToBank` to return its `AuthResult`, then the code carries on with the card.

6. The contact path (in the kernel's own order)

One stage = one native call = one `EmvStep`. Each `runContact*` method calls the kernel. On success it moves on with `advanceContactStage(NEXT)`; otherwise it ends the path with `finishContactStage()`, which releases the listener, closes the readers and reads the result.

#	EmvStep	PAX method	Native call	Asks a person?
0	`TERMINAL_INITIALIZATION`	`prepareKernel` → `preTransProcess`	`EMVCoreInit`, `EMVAddApp`	—
1	`SEARCH_CARD`	`onSearchCard` → `PaxCardSearch`	polling	—
2	`APPLICATION_SELECTION`	`runContactAppSelect`	`EMVAppSelect`	Application choice
5	`READ_APPLICATION_DATA`	`runContactReadAppData`	`EMVReadAppData` (GPO + records)	—
6	`SET_TRANSACTION_DATA`	`runContactSetTransactionData`	9C, `CARD_READ`	—
7	`OFFLINE_DATA_AUTHENTICATION`	`runContactCardAuth`	`EMVAddCAPK`, `EMVCardAuth`	—
8	`PROCESS_RESTRICTIONS`	`runContactStartTransaction`; then 13 if the card goes online, else the stage ends	`EMVStartTrans` = PR + CVM + TRM + TAA + 1st GAC	PIN
13	`ONLINE_AUTHORIZATION`	`onOnlineAuthorization` → `askTheBank` → `runContactOnline`; then 14	— (the bank)	—
14	`ISSUER_AUTHENTICATION`	`runContactIssuerAuthentication`; then the stage ends	`EMVCompleteTrans` = ARPC + scripts + 2nd GAC	—



- **When the sale goes to the bank.** When the card answers with an ARQC; a refund asks for an ARQC like a sale. The exception: a TC (`RESULT_OFFLINE_APPROVED`) on this online-only terminal is sent without an ARQC, and the bank's answer is the result. An AAC is a decline.
- **Each step does its own work, then hands over.** `PROCESS_RESTRICTIONS` → `ONLINE_AUTHORIZATION` → `ISSUER_AUTHENTICATION`, each in its own hook. `finishContactStage` runs in the last step that works, or where the path stops.

- **The 2nd GENERATE AC always follows the bank** (AET-146), even when the bank declined or did not answer. Only a cancel while waiting for the bank skips it: `goToStep(ISSUER_AUTHENTICATION)` returns `false` and the stage ends.
- **Scripts** run inside `EMVCompleteTrans`, so in contact `SCRIPT_PROCESSING` is a label, not a stage.
- `TERMINAL_RISK_MANAGEMENT` and `TERMINAL_ACTION_ANALYSIS` are not published: they have no native call of their own (they run inside `EMVStartTrans`).
- **The outcome** is reported once: `completeApproved` / `completeDeclined` → `finishApproved` / `finishDeclined` (no completion step).

7. النتيجة والنهاية الآمنة

الـ EMV يبلّغ بالنتيجة (`publish(APPROVED / DECLINED)` → `finishApproved` / `finishDeclined` ، و `finishError` ← `fail`)، والـ `Transaction` هي التي تنهي العملية. وقبل أن يصبح الجهاز متاحًا، يُغلق كل طلب أُرسل إلى البنك: **تأكيد** (acknowledgement) أو **إلغاء** (reversal).

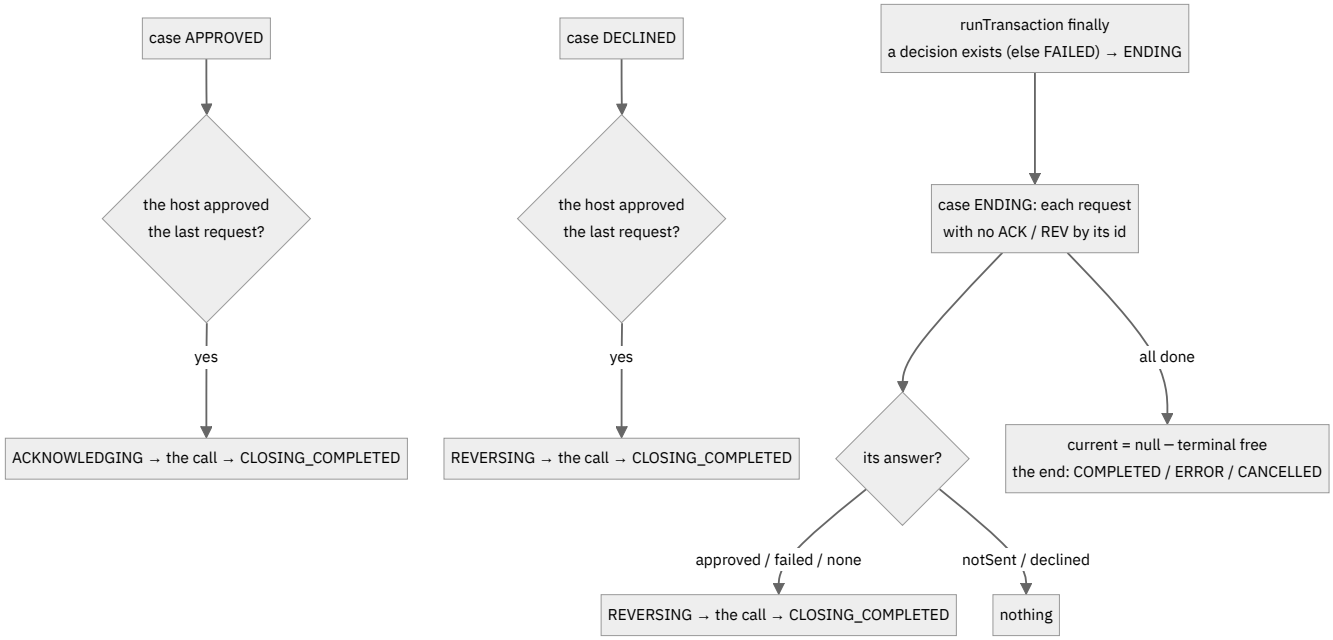
- **القرار نفسه يُغلق آخر طلب:** `case APPROVED` ← تأكيد، و `case DECLINED` ← إلغاء، بشرط أن البنك وافق على آخر طلب (إن رفض البنك فلا شيء). في هذه اللحظة رد البنك معروف، والبطاقة قررت، والـ reader مغلق.
- **`ENDING` يُغلق الباقي:** كل طلب لا توجد له `ACKNOWLEDGING` / `REVERSING` بالـ id نفسه في الخطوات يُلغى إن كان البنك ربما وافق (موافقة، `failed` ، لا رد). هذا يغطي الإلغاء (قد يأتي والبنك ما زال يرد)، و `FAILED` ، وطلبات محاولة سابقة. ثم يتحرر الجهاز وتُسلَّم النهاية.

البنك	البطاقة في الـ 2nd GAC	النتيجة	عند البنك بعدها
وافق	TC	APPROVED → COMPLETED	تأكيد
وافق	AAC	DECLINED → COMPLETED	إلغاء
رفض	—	DECLINED → COMPLETED	لا شيء
لا رد / انتهت المهلة (`failed`)	AAC	ERROR	إلغاء (ربما وافق)
لم يُرسل (`notSent`)	AAC	ERROR	لا شيء (لم يصل)

بالتفصيل: ماذا يقول الـ kernel

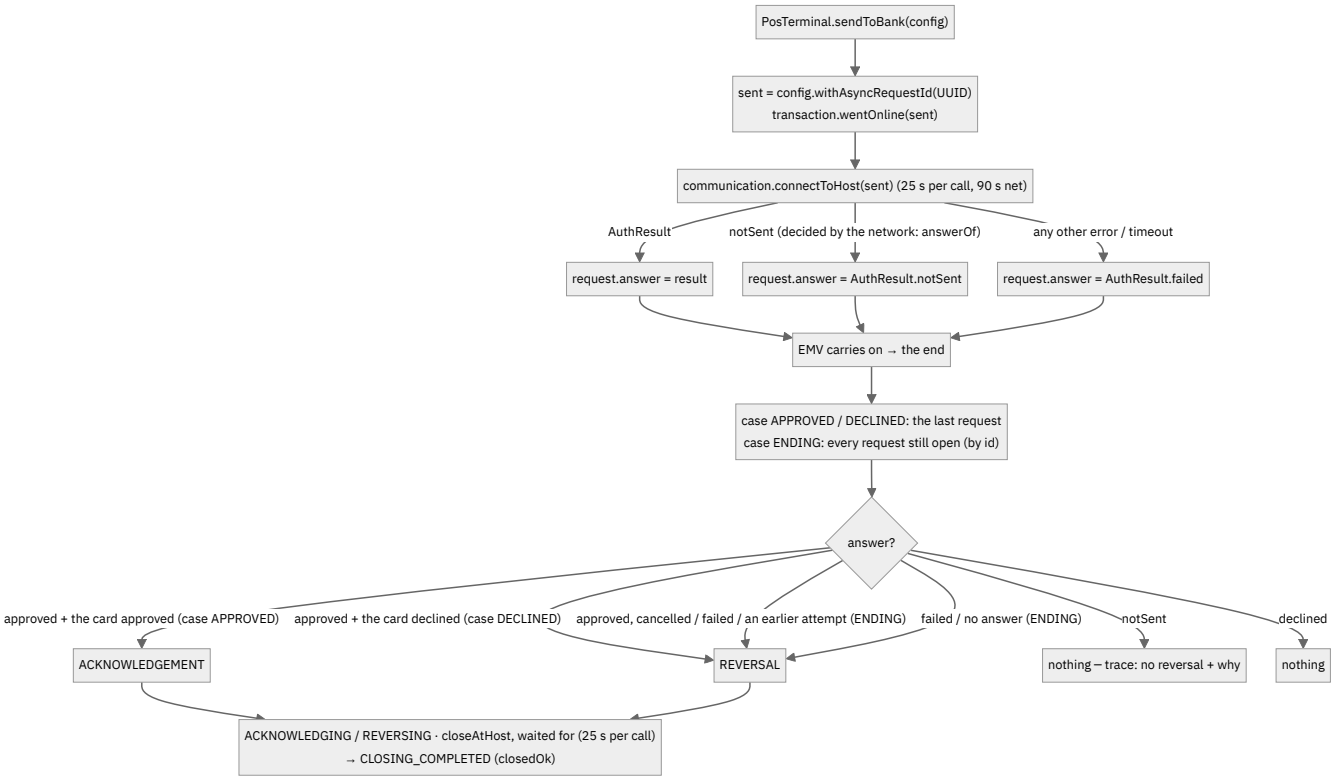
الـ host	البطاقة في 2nd GAC	`TransResultEnum`	النهاية	مع النهاية
APPROVE	TC	`ONLINE_APPROVED`	APPROVED → COMPLETED	acknowledge
APPROVE	غير ذلك	`ONLINE_CARD_DENIED`	DECLINED → COMPLETED	reverse
FAILED (لا رد / انتهاء المهلة / لم يُرسل)	أي نتيجة (الجهاز يطلب AAC)	`ONLINE_FAILED` / `FAILED` `_CARD_APPROVED`	ERROR "Online failed: <<step>>: ..."	reverse، إلا إذا كانت `notSent`
DENIAL	—	`ONLINE_DENIED`	DECLINED → COMPLETED	لا شيء

> ⚠ **مخاطر مفتوحة:** الإلغاء أثناء انتظار رد الـ host ينهي العملية بـ CANCELLED حتى لو وافق الـ host وأعطت البطاقة TC، وفي هذه الحالة يُرسل إلغاء (reversal) لذلك الطلب. يُرسل الإلغاء مرة واحدة: إذا فشل يُسجَّل في الـ log والـ trace فقط، دون سجل محفوظ لإعادة الإرسال (BACKLOG 0.15).



كيف يُقرَّر التأكيد أو الإلغاء لكل طلب

كل طلب أُرسل يُسجَّل لحظة إرساله (`wentOnline`) ومعه رده و id. القرار يُغلق آخر طلب، و `ENDING` يُغلق الباقي، والـ id هو الذي يربط بينهما:



من يقرر `notSent` ؟ الـ network وحده

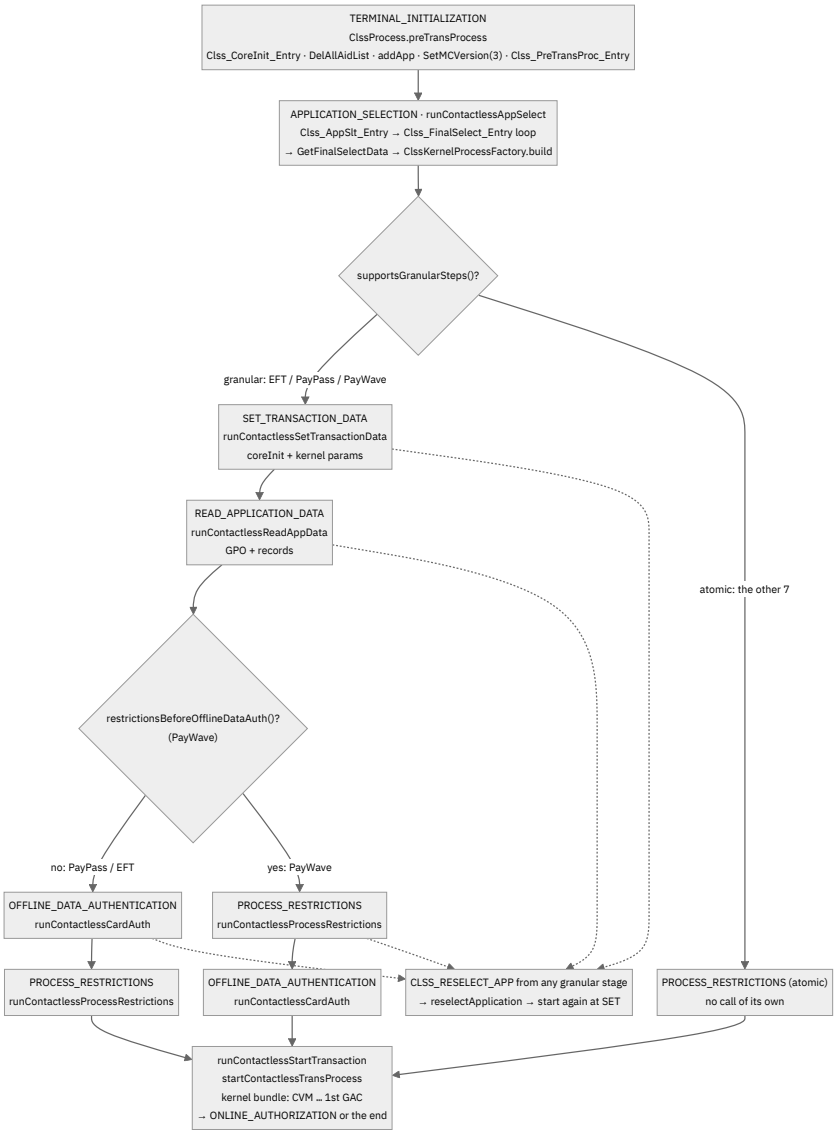
أين فشل	النتيجة	إلغاء؟
قبل الإرسال: لا مرجع للاسترداد، لا EMV result / مفتاح / token، فشل التشفير	`notSent`	لا
`Token refresh: …` أو `Exchange: …`	`notSent`	لا
`Sale:` لا اتصال (`ConnectException` / `Unknow` `nHost`) أو HTTP 401 / 403	`notSent`	لا
`Sale:` انتهاء المهلة، انقطاع، 4xx، 5xx، envelope آخر تالف	`failed`	نعم

المهلة	القيمة	ملاحظة
كل نداء دفع (OkHttp `callTimeout`)	25 ث	token + exchange + sale ≤ 75 ث؛ للدفع فقط، لا لتنزيل الـ params
`PosTerminal.sendToBank`	90 ث	شبكة أمان فوق الـ 75
`closeAtHost` (تأكيد / إلغاء)	60 ث	شبكة أمان؛ الحد الفعلي 25 ث للنداء

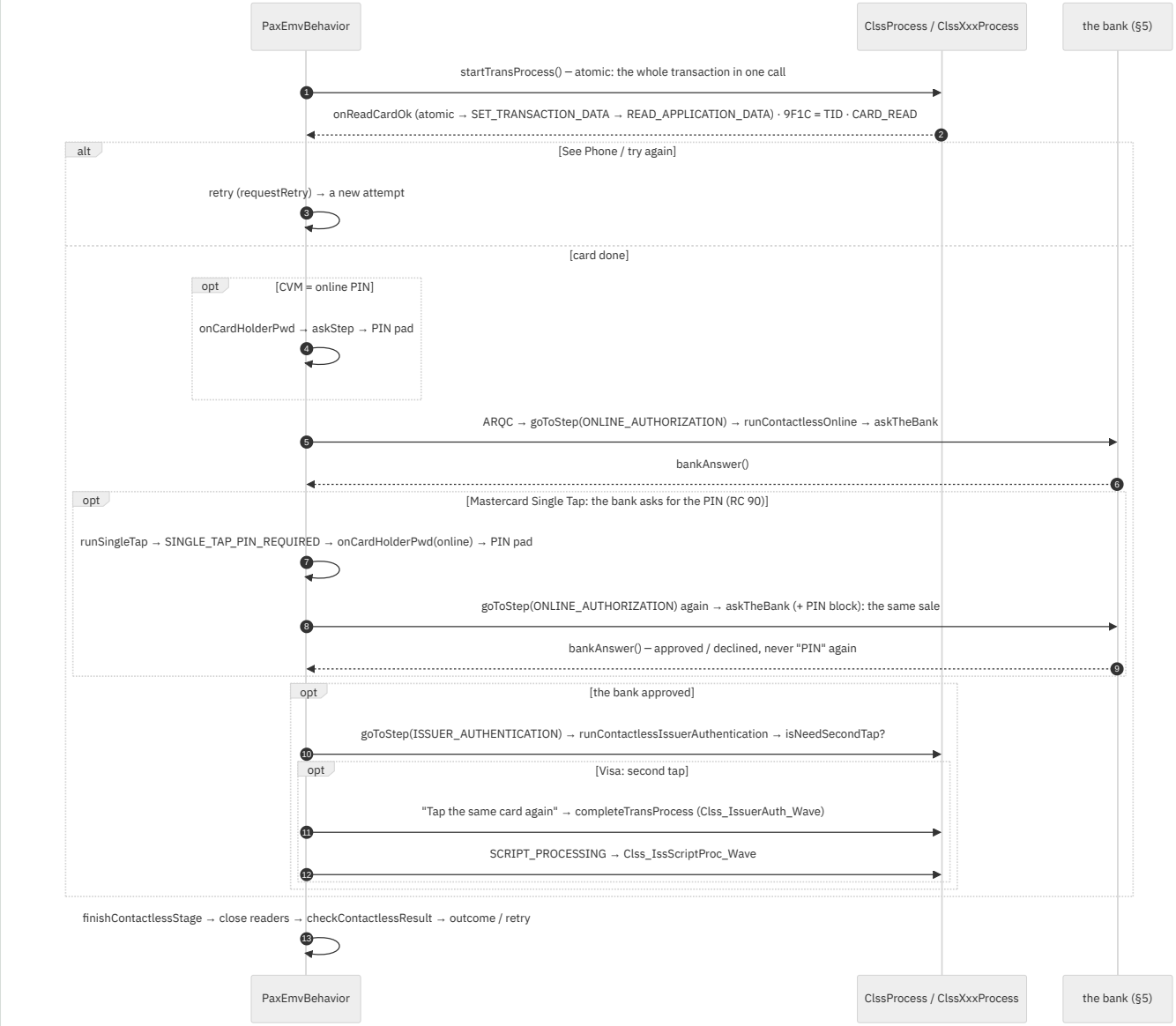
- ⚠ **مهلتا الـ 90 ث و 60 ث لا تقطعان الانتظار:** `connectToHost` / `acknowledge` / `reverse` مبنية بـ `fromCallable` بلا `subscribeOn` ، فتعمل على الـ transaction thread نفسه، و `timeout` لا تعود قبل أن ينتهي النداء (تجربة: مهلة 500 ms انتظرت 3124 ms). الحد الفعلي هو `callTimeout` (25 ث لكل نداء). إن جاء رد بعد المهلة يُرمى ويُعامل كـ `failed` ، فيُلغى الطلب (آمن ماليًا). الإصلاح (`subscribeOn(io)`) يحتاج نقاشًا: النداء يستمر في الخلفية بعد المهلة.
- نداءات الدفع تستخدم الـ adapter المتزامن (`createSynchronous()`)، وإلا يبتلع Retrofit الخطأ حين يقطع `callTimeout` النداء، فيبقى الجهاز مشغولًا للأبد (أُصلح في `1a97dee`).
- ⚠ التأكيد / الإلغاء يُرسل مرة واحدة: إن فشل يُسجَّل في الـ trace فقط، بلا سجل محفوظ لإعادة الإرسال (BACKLOG 0.15).
- يظهر اسم المرحلة في رسالة الـ ERROR وفي الـ trace: `Online Failed: Exchange: HTTP 403 Forbidden` .
- الـ trace (tag `HOST`): `Acknowledgement <id> sent` أو `… failed — not retried: <why>` ، `REVERSAL for request <id>: approved, not completed` ، `REVERSAL …: no usable host` ، `answer — Sale: timeout` ، `no reversal …: the host could not have approved it —` `Exchange: HTTP 403 Forbidden`.
- **`pinRequired`** (Mastercard Single Tap، RC 90، §8): ليس موافقة ولا فشلًا، فلا يُلغى. الإعادة بالـ PIN هي الطلب نفسه (`OnlineRequest` و `asyncRequestId` نفسهما، والرد يُستبدل)، فيُغلق مرة واحدة.
- لا يتغير شيء بالنسبة للبطاقة: `failed` و `notSent` كلاهما `RESULT_ONLINE_FAILED` (2nd GENERATE AC بـ AAC). الفرق في الإلغاء فقط.
- ⚠ لم يُختبر على الجهاز ولا مع الـ host. احتياج الإلغاء والتأكيد إلى `mToken` افتراض.

8. مسار الـ contactless

طريقة التشغيل نفسها و `askStep` نفسها، لكن الـ kernel يُختار بعد اختيار التطبيق (`ClssKernelProcessFactory`). نوعان: **granular** (PayPass و PayWave و EFT) ولكل مرحلة استدعاء native مستقل، و**atomic** (Amex و JCB و Mir و PBOC و Pure و RuPay و DPAS) وفيها استدعاء واحد يعمل كل شيء.



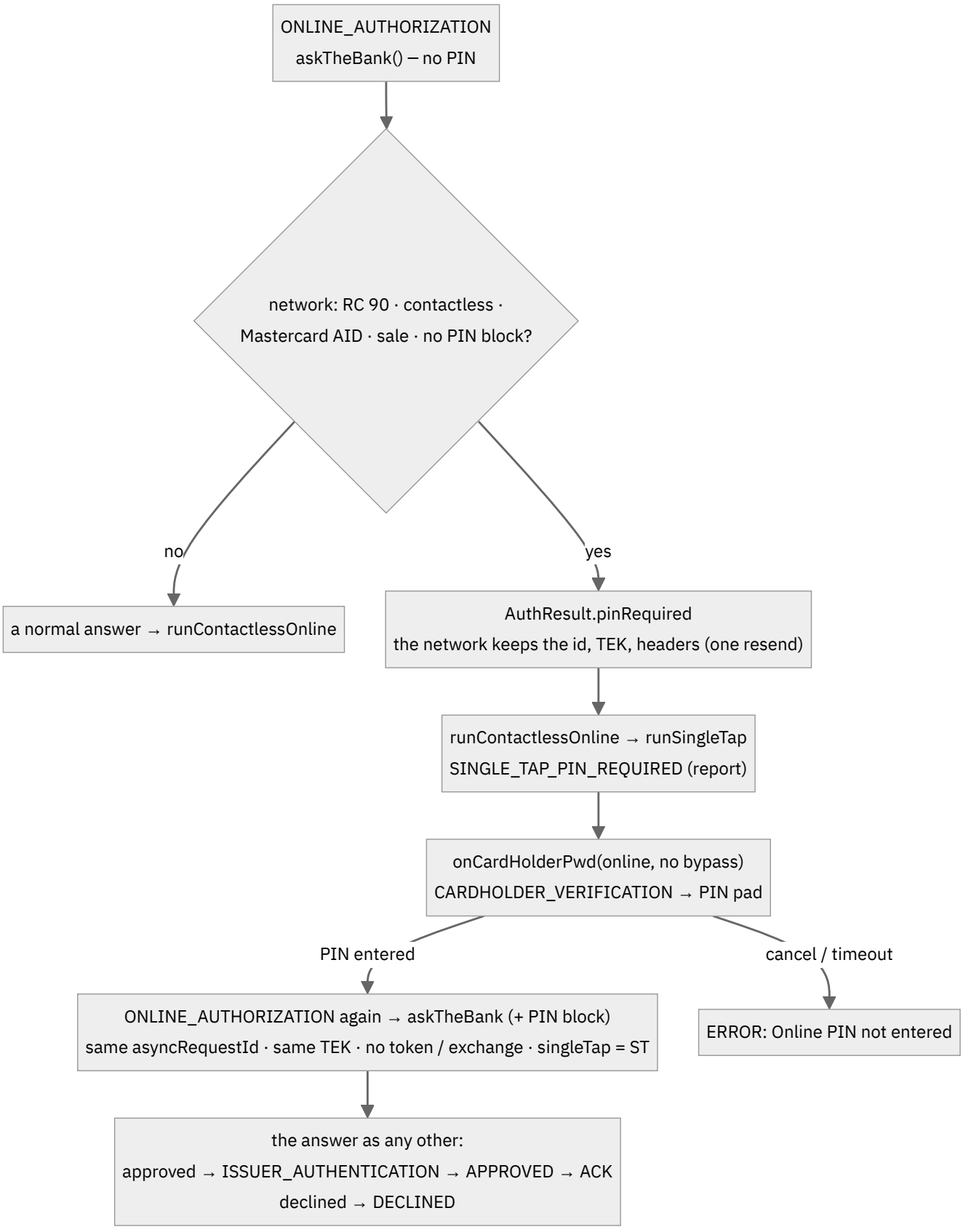
بعد المراحل: `startContactlessTransProcess`



- **الترتيب يتبع الـ kernel، لا الأرقام:** في الـ atomic تُعلَن `READ_APPLICATION_DATA` و `SET_TRANSACTION_DATA` من الـ callbacks في اللحظة التي يقول فيها الـ kernel إنها حدثت، ولا تُعاد القراءة (الـ hooks لا تعمل شيئًا للـ atomic).
- إذا لم يوافق البنك، فلا issuer authentication ولا 2nd GENERATE AC: البطاقة غادرت الحقل.
- ⚠ رسالة الـ tap الثاني لا تنتظر البطاقة قبل `Clss_FinalSelect_Entry` (BACKLOG 1.4).

Mastercard Single Tap: البنك يطلب الـ PIN (`35ce685`)

بيع contactless بـ Mastercard تحت حد الـ CVM يذهب إلى البنك بلا PIN. قد يرد البنك بـ "PIN" (RC 90 "required"). عندها يأخذ الجهاز الـ PIN، **بلا tap ثانٍ**، ويرسل البيع نفسه مرة أخرى، كما كان التطبيق القديم يفعل.



- **لا حلقة، بثلاثة حراس منفصلين** (لكل واحد test يفشل بدونه): الـ network: RC 90 مع PIN رفض عادي، لا "PIN" مرة ثانية. الـ core: الإعادة هي الطلب نفسه، فتُغلق مرة واحدة، و `pinRequired` لا يُلغى أبدًا. PAX: الـ PIN يُطلب مرة واحدة في المحاولة (`tx.singleTapPinEntered`)، وطلب ثانٍ يُنهي العملية DECLINED.

- **مفتاح الـ PIN:** يُرسل مع كل exchange، بـ PIN أو بدونه (`pinKeyToSend`)، ومفتاح واحد لكل محاولة (`ensureOnlinePinKey`) في `addOnlineRequestData`). فالـ PIN block في الإعادة مبني بالمفتاح الذي استلمه البنك في الـ exchange الأول.
- الـ PIN pad والإعادة لا يمسكان الـ lock الخاص بخطوات العملية، فلا ينتظر `cancel` .
- ⚠ **مفتوح:** كيف يصل RC 90 (HTTP 200 أم 4xx)؟ لو جاء 4xx يصبح `failed` + إلغاء، ولا يبدأ Single Tap (BACKLOG 4.11). و posEntryMode في الإعادة 071 (افتراض). ⚠ لم يُجرَّب على الجهاز ولا مع البنك.

EmvStep	PayPass (`ClssPayPassProcess`)	PayWave (`ClssPayWaveProcess`)	EFT (`ClssEFTProcess`)
`SET_TRANSACTION_DATA`	`SetParam_M` ، `Clss_CoreInit_MC` FinalSelectData + `setPayPass` ، `C Param` (DF8117/18/19/1F، limits)	ReadParam ، `CoreInit_Wave` `SetVisaAi` + (`aucTmCap` = 9F33) `SetTransData_Wav` ،`dParam_Wave e`	ReadParam + + `CoreInit_EFT` `SetTransData_EFT` + AidParam
`READ_APPLICATION_DATA`	`Clss_R` + `Clss_InitiateApp_MC` `eadData_MC`	GPO +) `Clss_Proctrans_Wave` (records + AC type	`Clss_Read` + `Clss_InitApp_EFT` `AppData_EFT`
الترتيب بعدها	ODA ثم PR	**PR ثم ODA**	ODA ثم PR
`OFFLINE_DATA_AUTHENTICATION`	تحميل CAPK و revocation فقط (الـ ODA داخل `TransProc`)	`Clss_CardAuth_Wave` (عند TC وليس استردادًا، أو حسب TTQ)	`Clss_Car` + CAPK + `PreTAAProc dAuth_EFT`
`PROCESS_RESTRICTIONS`	لا يوجد استدعاء	`Clss_ProcRestric_Wave`	`Clss_ProcRestrict_EFT`
الـ bundle	`Clss_TransProc_MC_MChip` / `_M` = `ag` ODA+PR+CVM+TRM+TAA+GAC، ثم see phone	`Clss_GetCvmT` : `genTransResult` AC type + `ype_Wave`	`Clss_Post` + `Clss_CVMProc_EFT` `TAAProc_EFT`
2nd tap / completion	`completeTransProcess`	`Clss_IssuerAuth_W` + FinalSelect `ave`	`Clss_CompleteOnlineTrans_EFT` ⚠ لا يُستدعى
`SCRIPT_PROCESSING`	default	`Clss_IssScriptProc_Wave`	default

النتيجة: `checkContactlessResult`

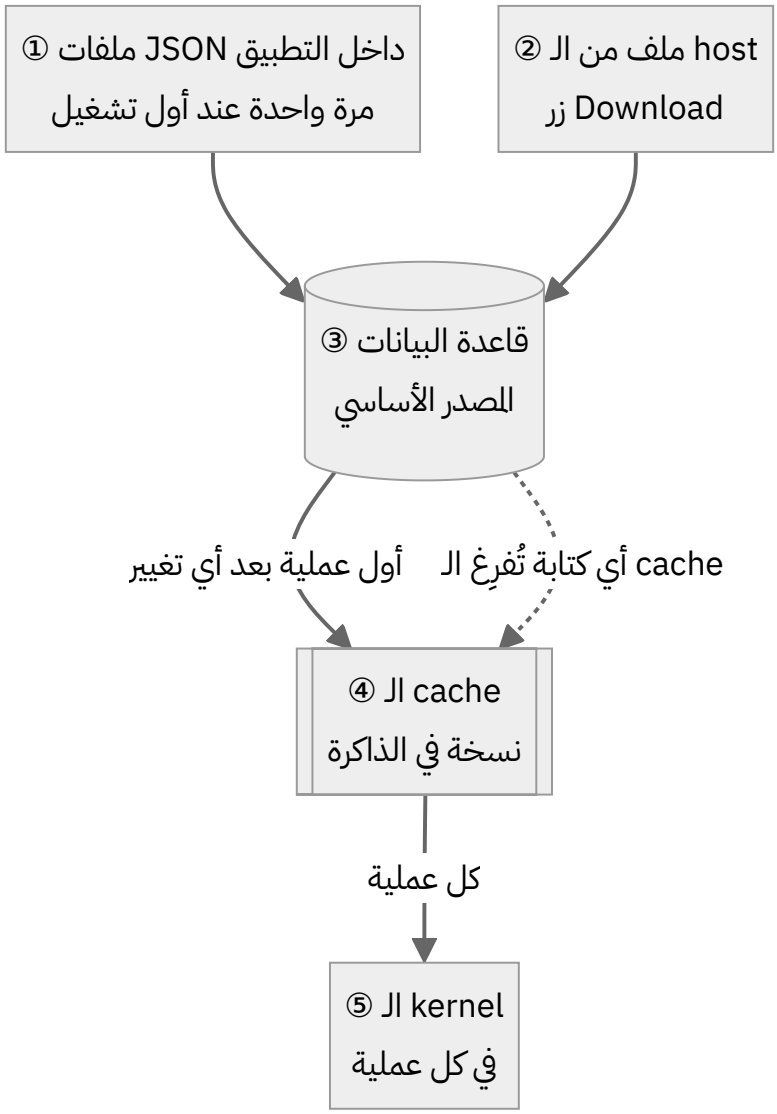
الحالة	الدالة	النهاية
`RESULT_CLSS_SEE_PHONE` (أي ret)	`seePhone` (+ `clsLastNeedSeePhone`)	إعادة المحاولة بالـ CONTACTLESS، وبعد 3 مرات رفض
`RESULT_TRY_AGAIN` (أي ret)	`tryAgain`	إعادة المحاولة بالطريقة نفسها، وبعد 3 مرات رفض
رد البنك `pinRequired` (قبل قراءة النتيجة)	`runSingleTap`	الـ PIN ثم `ONLINE_AUTHORIZATION` مرة ثانية
OK + `ONLINE_APPROVED`	`onlineApproved`	APPROVED
OK + `OFFLINE_APPROVED`	`offlineApproved`	"ERROR "Online Failed
OK + غير ذلك	`offlineDenied(CLSS_DECLINE)`	DECLINED
`TRY_ANOTHER_INTERFACE` / `CLSS_USE_CONTACT`	`tryAnotherInterface`	إعادة المحاولة بالـ CHIP
`ONLINE_DENIED` / `ONLINE_CARD_DENIED` / `OFFLINE_DENIED`	كما في الـ contact	DECLINED
`ONLINE_FAILED`	`onlineFailed`	ERROR

`IContactlessCallback`	من يستدعيها	ماذا تفعل
`onReadCardOk`	الـ kernel بعد القراءة	atomic: `goToStep(SET_TRANSACTION_DATA)` ثم `goToStep(READ_APPLICATION_DATA)`؛ `APPL ICATION_SELECTED`؛ 9F1C؛ `CARD_READ`. يأتي قبل أن يقرر الكارت، فيظهر `CARD_READ` أيضًا قبل See Phone أو الرفض.
`confirmCard`	لا أحد (الـ kernel لا يستدعيها)	ترجع `CONTACT_OK` فقط؛ باقية لأن الواجهة تطلبها
`onWaitAppSelect` / `onCardHolderPwd`	مشتركة مع الـ contact	`askStep`
`startOnlineProcess`	لا أحد	ترجع `bankAnswer()` بشكل PAX فقط؛ الذهاب إلى البنك هو `ONLINE_AUTHORIZATION`
`onDetect2ndTap`	مرحلة الإكمال	رسالة + `SEARCH_CARD` (للعرض فقط، دون بحث ثانٍ)
`needSeePhone`	الـ kernel في الـ tap التالي (Amex / Mir / Pure)	تعيد `clsLastNeedSeePhone` ثم تصفّرها
`onRemoveCard`	PayPass / PayWave	تسجيل في الـ log فقط

9. الـ EMV params: مصدرها، ومكان حفظها، ومتى تصل إلى الـ kernel

ما هي الـ params؟ هي القواعد التي يحتاجها الـ kernel ليتخذ قراره في كل عملية: البطاقات المقبولة (الـ AIDs)، والمفاتيح التي يُتحقَّق بها من أن البطاقة أصلية (الـ CAPKs)، والحدود المالية (مثل المبلغ الذي يُطلب بعده PIN في الدفع اللاتلامسي). بدونها لا يستطيع الـ kernel العمل.

الصورة العامة



دورة حياة الـ params في 5 خطوات

1. **عند أول تشغيل للتطبيق:** تُملأ قاعدة البيانات من ملفات JSON المرفقة بالتطبيق، حتى يكون هناك ما يعمل به. يحدث ذلك مرة واحدة فقط.
2. **عند الضغط على Download:** يرسل الـ host ملفًا (XML للـ contact وآخر للـ contactless)، فيُقرأ ويُكتب في قاعدة البيانات بدلًا من القديم.
3. **قاعدة البيانات هي المصدر الأساسي:** القيم الفعلية للـ params موجودة فيها.
4. **الـ cache نسخة سريعة:** أول عملية بعد أي تغيير تقرأ قاعدة البيانات كاملة مرة واحدة وتحتفظ بها في الذاكرة، والعمليات التالية تأخذ من الذاكرة مباشرة.

5. **الـ kernel لا يحتفظ بشيء:** في **كل** عملية تُحمَّل إليه الـ params من الـ cache من جديد، وتُمسح منه بانتهاء العملية.

مثال

غيّر الـ host حدّ طلب الـ PIN في الدفع اللاتلامسي من 600 إلى 1000 جنيه:

- **Download:** أصبحت القيمة في قاعدة البيانات 1000، وأُفرِغ الـ cache.
- **أول عملية بعدها:** يُعاد ملء الـ cache من قاعدة البيانات (1000)، ويستلمها الـ kernel.
- **عملية tap بمبلغ 800 جنيه:** تتم دون طلب PIN ✓

ماذا تعرض شاشة EMV Params؟

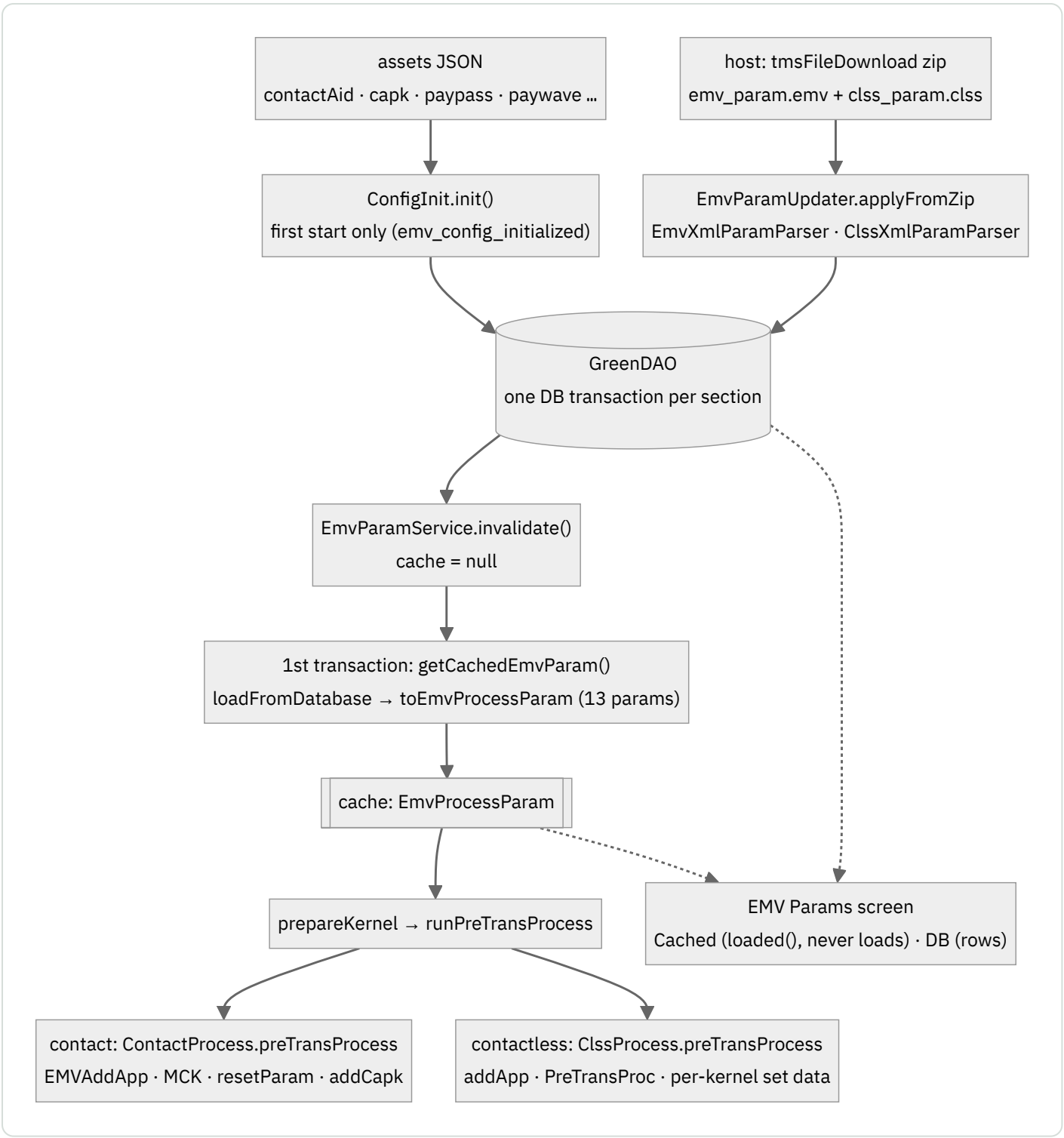
- **DB:** القيم المحفوظة (المصدر الأساسي).
- **Cached:** القيم التي سيستلمها الـ kernel الآن. إن كانت فارغة فمعنى ذلك أنه لم تتم أي عملية منذ آخر تغيير، وليس أن هناك خطأ.

قواعد مهمة

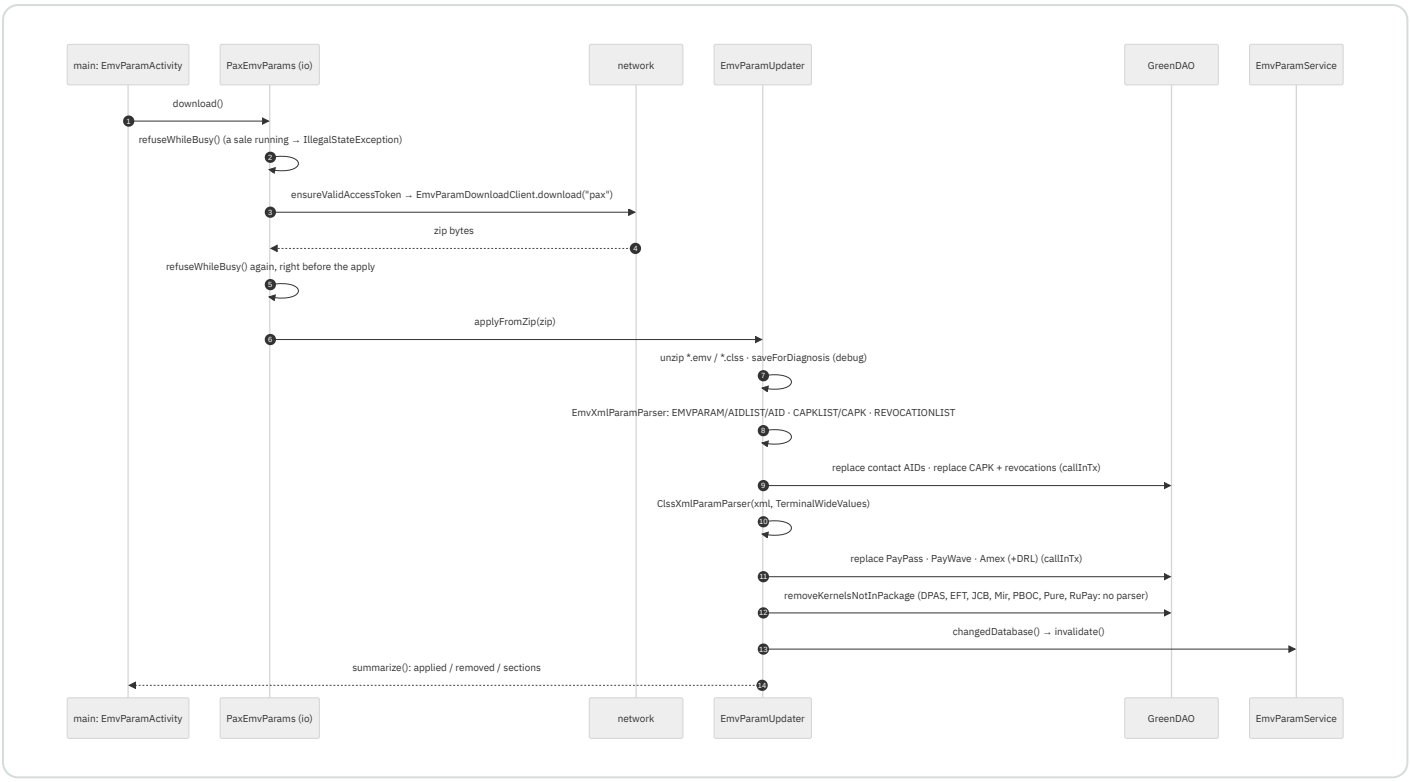
- **الـ contact:** إذا لم يتضمن ملف الـ host جزءًا معينًا، تبقى القيم القديمة كما هي. **الـ contactless:** ملف الـ host هو المرجع الكامل، وأي نوع بطاقة غير موجود فيه يُحذف.
- بيانات الجهاز (MCC، MID، TID، اسم التاجر) **لا** تأتي من ملف الـ host.
- ⚠ لا يوجد فحص لصلاحية الـ CAPKs: 76 من 88 في ملفات الـ JSON منتهية الصلاحية.

التفاصيل التقنية (للمراجعة)

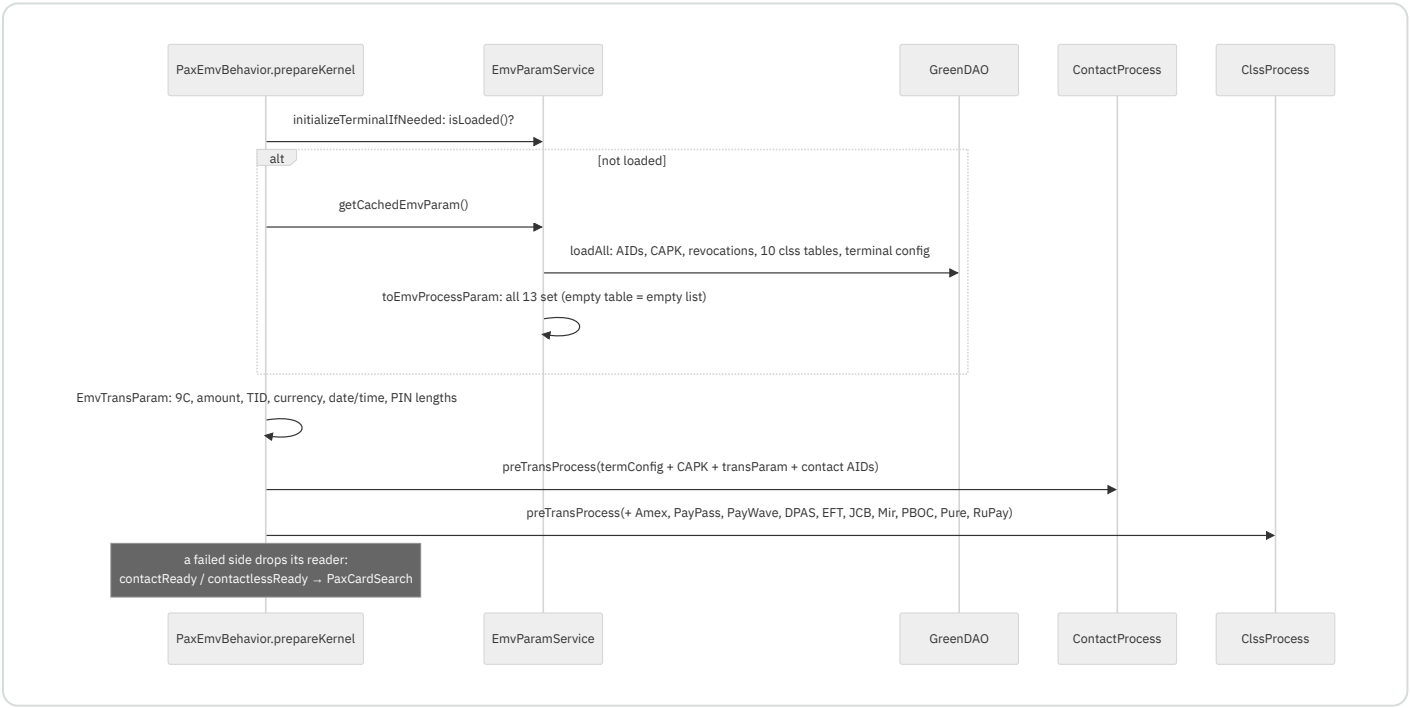
المسار بأسماء الكلاسات: يكتب `ConfigInit` (أول تشغيل) و `EmvParamUpdater.applyFromZip` (التنزيل) في GreenDAO؛ وتُفرغ `EmvParamService.invalidate()` الـ cache؛ `getCachedEmvParam()` → `loadFromDatabase` → `toEmvProcessParam` (param 13)؛ `prepareKernel` → `preTransProcess` للـ contact والـ contactless.


assets JSON
contactAid · capk · paypass · paywave …
host: tmsFileDownload zip
emv_param.emv + clss_param.clss
ConfigInit.init()
first start only (emv_config_initialized)
EmvParamUpdater.applyFromZip
EmvXmlParamParser · ClssXmlParamParser
GreenDAO
one DB transaction per section
EmvParamService.invalidate()
cache = null
1st transaction: getCachedEmvParam()
loadFromDatabase → toEmvProcessParam (13 params)
cache: EmvProcessParam
prepareKernel → runPreTransProcess
EMV Params screen
Cached (loaded(), never loads) · DB (rows)
contact: ContactProcess.preTransProcess
EMVAddApp · MCK · resetParam · addCapk
contactless: ClssProcess.preTransProcess
addApp · PreTransProc · per-kernel set data

التنزيل خطوة بخطوة



أول عملية بعد التنزيل

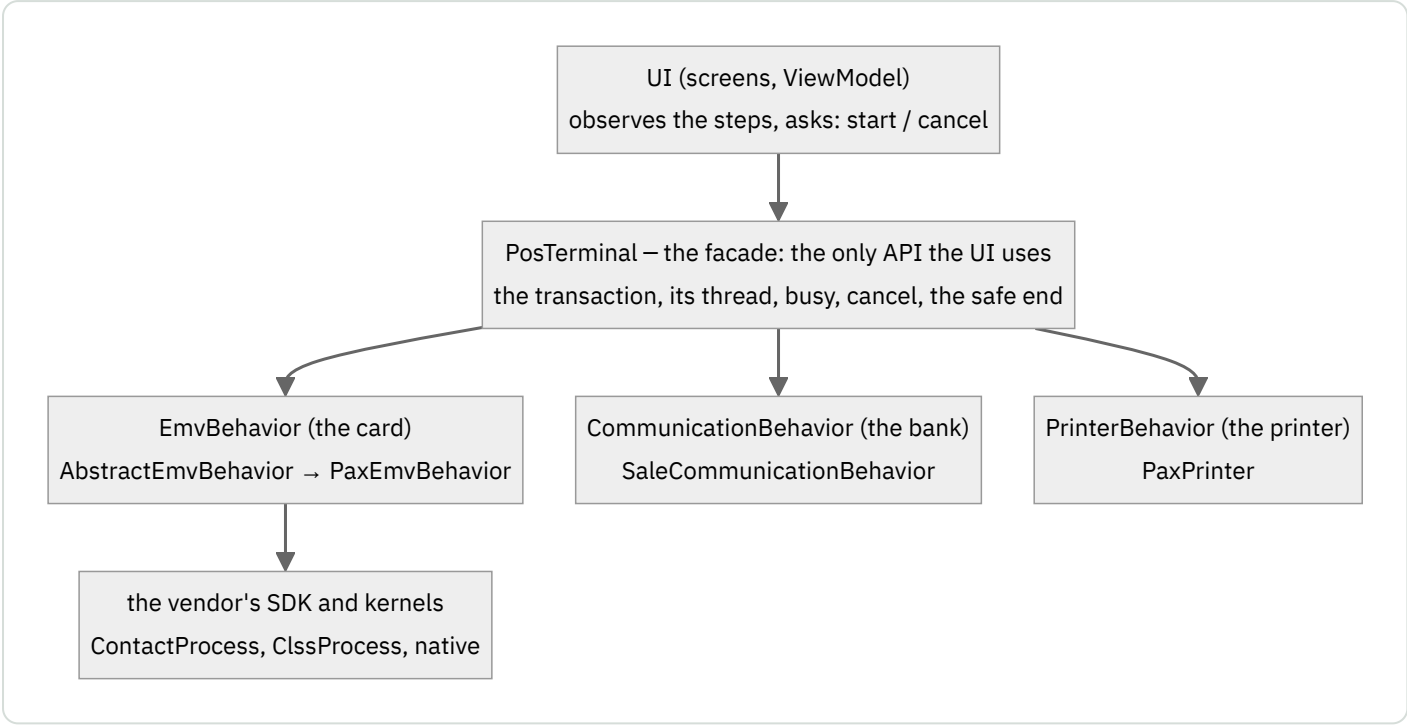


الـ param	أين يدخل الـ kernel
الـ contact AIDs، وأول AID كقيمة افتراضية	`ContactProcess.preTransProcess` (`EMVAddApp`، `setEmvAndMCKPara`m)
9F33 / 9F40 / 9F35 الخاصة بالـ AID المختار	`emvSetParam` → `resetParam` (داخل `EMVReadAppData`)
CAPK و revocation	contact: `cardAuthentication` → `addCapk`؛ contactless: مرحلة الـ ODA
الـ contactless AIDs والحدود	`ClssEntryAddAid.addApp` + `Clss_PreTransProc_Entry`
TTQ، 9F33، floor، CVM limit، DF81xx	`setTransactionData` لكل (granular) kernel أو داخل الاستدعاء الواحد (atomic)
الـ TID في field 55 للـ contactless	`onReadCardOk` → `setTlv(9F1C)`

- `OTHERCONFIGURATION` (الـ TID و الـ MCC وبيانات التاجر) لا يُؤخذ من الـ XML، وهذا قرار مقصود.
- يُتجاهل أي CAPK بلا RID أو KeyID صحيح. ⚠ لا يوجد فحص للصلاحية ولا للـ checksum.
- أعاد `5755adb` الدوال الـ 12 `setXxxParam` الناقصة في `loadFromDatabase`، وهو إصلاح الـ crash في `preTransProcess`، ويحميه `EmvParamServiceCacheTest`.

10. مبادئ البرمجة في المشروع (ببساطة)

كل مبدأ في سطر: معناه ببساطة، وأين يظهر في الكود. لا شيء هنا للزينة؛ كل واحد يحل مشكلة حقيقية في الدفع.



المبدأ / النمط	ببساطة	أين في الكود
Layers + Facade	طبقات، كل طبقة تكلم التي تحتها فقط. والـ UI يرى بابًا واحدًا يخفي ما خلفه	`PosTerminal` هو الباب الوحيد للـ UI (`startTran` / `saction` / `acceptCard` / `steps()` / `cancel`). ليس طبقة تمرير فارغة: يملك العملية فعلًا (القاعدة 5 تمنع الطبقات التي تمرّر فقط)
Strategy / Polymorphism	واجهة واحدة وأكثر من تنفيذ، يُختار واحد منها	`EmvBehavior` (PAX / Ingenico / Fake)، `Commu`nicationBehavior، `PrinterBehavior`
Open / Closed	نضيف بدل أن نعدّل	vendor جديد = flavor جديد بـ `TerminalFactory` وكلاساته، والـ core لا يتغيّر
Template Method	الكلاس الأب يحدد الترتيب، والابن يملأ الخطوات	`AbstractEmvBehavior` : `run` و `goToStep` والـ dispatch، و 16 `onXxx` يملؤها الـ vendor؛ `askThe`nticationBehavior `final` `Bank()`
Observer	من يهمه الأمر يشترك، ويصله كل جديد	`Transaction.steps()` (`ReplaySubject`) والشاشة مشتركة؛ `EmvEngine.emvSteps` و `EmvSt`epObserver؛ `TransactionStepObserver`
Callback	تعطي جهة "رقمًا تتصل بك عليه"	`BankRequest` (§5)، `CardSearchListener` ، `I`nteractionCallback و `askStep` (§4)
Adapter	محوّل بين شكلين لا يعرف أحدهما الآخر	`PaxHostAnswer` (`AuthResult` → شكل PAX)؛ `ContactProcess.EmvCallBackListener` (native → listener)؛ `PaxEmvBehavior` بصفته `I`nteractionCallback
Factory / Builder	مكان واحد يقرر كيف يُنشأ الشيء	`TerminalFactory` (لكل flavor)، `ClssKernelPr`ocessFactory (لكل kernel) لكل scheme)، `AuthResu`lt.approved / declined / failed / notSen`t`، `TransactionStepEvent.builder(...)`
Dependency Injection (يدوي)	الكلاس يأخذ ما يحتاجه من الخارج ولا يُنشئه	`PaxTerminal` → `super(behavior, communic`ation, printer)؛ و `attach(this::sendToBa`nk) للدائرة الوحيدة
Dependency Inversion (الـ D في SOLID)	الطبقة العليا تعتمد على واجهة، لا على كلاس محدد	`AbstractEmvBehavior` يعرف `BankRequest` ، لا `PosTerminal` ؛ الـ core يعرف `CommunicationBeh`avior، لا Retrofit
Single Responsibility	كل كلاس له شغلة واحدة	`PaxCardSearch` (البحث)، `PaxPinEntry` (الـ PIN)، `PaxHostAnswer` ، `PaxReaders` ، `PaxCar`dData، `PaxSetup` ، `PaxKernelSession`
Single source of truth	كل حالة لها مالك واحد ونسخة واحدة	الجهاز مشغول = `PosTerminal.current` ؛ حالة العملية كلها = خطواتها (`Transaction.decision`)؛ خطواتها = `Transaction.steps` ()
Immutability / Value Object	الشيء لا يتغيّر بعد إنشائه؛ التعديل يعطي نسخة جديدة	`AuthResult` ، `TransactionStepEvent` ، `Tran`sactionConfig.`sactionConfig.withXxx(...)` يرجع config جديدًا
Fail closed	عند الشك نرفض، لا نوافق	لا رد = online failed؛ لا PIN = `USER_CANCEL` ؛ الـ callback يرد دائمًا (`setCallBackResult`)

11. استخدامنا كـ SDK (أي تطبيق يعمل integration)

الفكرة: أي تطبيق يستخدمنا للجزء بتاع الكارت والـ EMV. **أبسط عقد:** التطبيق يدّينا البنك بتاعه، ويسمع للـ steps. الـ `dispatchTransactionStep` (التأكيد والإلغاء) يفضل جوّانا، والتطبيق لا يكتبه.

الاتجاه	ماذا	كيف
التطبيق → إحنا	**البنك**	يكتب `CommunicationBehavior` : `connectToHo st(request)` إجباري (يبعت البيع ويرجّع `AuthResul t`)، و `acknowledge` / `reverse` اختياريين لو البنك بيدعمهم. الطلب يوصله وعليه الـ id (`request.g etAsyncRequestId()`) جاهز.
التطبيق → إحنا	الطباعة (اختياري)	`PrinterBehavior.print(lines)`
إحنا → التطبيق	**الـ steps**	`transaction.steps()` : يعمل subscribe ويعرض: `APPROVED` / `DECLINED` / `FAILED`، والنهاية `COMPLETED` / `ERROR` / `CANCELLED` (الجهاز متاح)، والباقي تقدّم.
التطبيق → إحنا	البدء والإلغاء	`pos.acceptCard(type, amount)` و `transact ion.cancel()`

```
// the integrator's bank — the only code they must write for the EMV part
class MyBank implements CommunicationBehavior {
    // required: send the sale, return the answer
    public Single<AuthResult> connectToHost(TransactionConfig request) { … }
    // optional
    public Completable acknowledge(TransactionConfig sent) { … }
    public Completable reverse(TransactionConfig sent) { … }
}

// their own screen (level B): start, observe, cancel
Transaction t = pos.acceptCard(TransactionType.SALE, amountMinor);
t.steps().subscribe(e → {
    switch (e.getStep()) {
        case APPROVED:  /* approved + receipt fields */ break;
        case DECLINED:  /* declined */ break;
        case FAILED:    /* the error */ break;
        case COMPLETED:
        case ERROR:
        case CANCELLED: /* done, terminal free */ break;
        default:        /* progress: CARD_READ, ONLINE_* … */ break;
    }
});
cancelButton.setOnClickListener(v → t.cancel());
```

- **ليه الـ switch بتاعنا يفضل جوّانا:** هو اللي بيحمي الفلوس. تأكيد لما الكارت يوافق، وإلغاء لما يرفض، وإلغاء في `ENDING` بعد cancel أو failure (الـ cancel ممكن ييجي والبنك لسه بيرد). لو كل تطبيق كتبه، واحد هينسى الإلغاء

بعد الـ cancel، فالفلوس تتخصم من العميل. جوّانا هو مكتوب مرة واحدة ومتغطّي بالـ tests.

- **الـ id بتاع التطبيق مش بتاعنا:** `sendToBank` بيسمّي الطلب (`asyncRequestId`) قبل ما يوصل للتطبيق، والتأكيد والإلغاء بيرجعوا بنفس الـ id. الـ EMV عمره ما بيشوفه.
- الـ switch اللي فوق في كود التطبيق للشاشة بتاعته بس. هو مش اللي بيحرّك العملية.

أبسط integration: بشاشة الـ search card بتاعتنا (level A)

التطبيق ما يكتبش شاشة خالص: يفتح شاشتنا بالمبلغ، ويستلم النتيجة.

```
// once, in their Application
class App extends Application implements TerminalProvider {
    private PosTerminal pos;

    public void onCreate() {
        super.onCreate();
        pos = TerminalFactory.create(this, new MyBank(), new MyPrinter());
    }

    public PosTerminal terminal() {
        return pos;
    }
}

// to take a payment: one launch, one result
ActivityResultLauncher<PaymentRequest> pay =
        registerForActivityResult(new EasyLinkPayment(), outcome → {
            // APPROVED / DECLINED / FAILED / CANCELLED,
            // the message, the masked PAN
        });
pay.launch(new PaymentRequest(TransactionType.SALE, amountMinor));
```

أسهل الخطوات عشان نطلّع الـ SDK والشاشة

#	الخطوة	ليه
1	`TerminalFactory.create(app, communicatio` `n, printer)`: البنك والطابعة يدخلوا من برّه. لو التطبيق ما بعتش حاجة، يفضل الـ default (`SaleComm` `unicationBehavior` و `PaxPrinter`).	النهارده `PaxTerminal()` بيعمل البنك والطابعة بنفسه، فالـ integrator مش هيقدر يدّينا البنك بتاعه. الـ constructor اللي بياخدهم موجود، بس `private` .
2	Android library module جديد (مثلًا `easylink-u` `i`) معتمد على `core` بس، فيه `SearchCardFragm` `ent` و `MainViewModel` (+ الـ Factory) و `Even` `t`، والـ layouts التلاتة (`fragment_search_card` و `item_entry_method` و `item_kv_row`) والـ strings بتاعتهم.	الشاشة النهارده جوّه `app` . نسمّي الـ resources بـ prefix (`easylink_`) عشان ما تتخانقش مع resources التطبيق.
3	interface اسمه `TerminalProvider` فيه `PosTer` `minal terminal()`. الـ `Application` بتاع التطبيق يعمله implement، و `EmvEnhanceApp` كمان.	ده الرابط الوحيد بين الشاشة والتطبيق. النهارده الشاشة بتعمل `((EmvEnhanceApp) getApplication()).` `getTerminal()`. وده interface ليه تطبيقين، فقاعدة 5 مش بتمنعه.
4	`PaymentActivity` (أو contract زي `EasyLinkPa` `yment`) شايل الـ fragment ويرجّع النتيجة بالـ Activity Result API.	النهارده زرار Done بيعمل `finish()` بس، والنتيجة ما بترجعش لحد.
5	الحاجات بتاعة الـ debug (الـ APDU trace على الشاشة وطباعته، ونافذة اختيار التطبيق حتى لو AID واحد) تشتغل في debug بس.	دي أدوات لينا، مش للـ integrator.
6	⚠ قبل ما نسلّم الـ SDK لأي حد: بوابة الـ release (BACKLOG §5): الـ PAN كامل و 55 field على الشاشة والإيصال، و logging `Level.BODY` ، و mToken حقيقي في الكود.	دي لو راحت لحد تاني، تبقى بيانات كروت متسربة.

الـ vendor (PAX) لسه بيتختار بالـ flavor و `TerminalFactory` ، فالـ integrator بياخد `core` و `vendor-` `pax` (ومكتباته) و `easylink-ui` . الكلام ده لسه اتجاه، ومفيش كود اتعمل له.

ملحوظة: الفرق مع `paxEmvBehaviorSplit`

الفرع الأقدم `paxEmvBehaviorSplit` (ومعه `-tests`) يعمل بالطريقة نفسها مع أسماء وتفاصيل أقدم: `authorizeOnline` / `Authorizer` / `PosTerminal.authorize` / `CommunicationBehavior.authorize`؛ و `START_ONLINE_PROCESS`؛ و `tx.lastAuth` في كل vendor؛ و `CLOSED` بدل `CLOSING_COMPLETED`؛ ولا `ENDING` (النهاية عبر `onEnd` وحلقة)؛ والـ switch تستدعيه أكثر من جهة؛ والـ network بسلسلة Rx (`flatMap`) بلا `callTimeout`. القائمة الكاملة في `doc/REFERENCE.md`.